

Module 1: Python Basics

I. Introduction to Jupyter

Jupyter is a freely available web application that enables creation and sharing of documents containing equations, live coding, visualizations, and narrative text. Jupyter provides an interactive computing environment that supports multiple programming languages, including Python, R, Julia, and more, but it shines brightest when used with Python. Jupyter revolves around notebooks, documents containing a mix of code, visualizations, narrative text, equations, and multimedia content. These notebooks allow users to create, share, and collaborate on computational projects seamlessly.

Why Jupyter?

Jupyter's popularity stems from its flexibility and ease of use. Regardless of your level of programming expertise, whether you're an experienced coder or embarking on your data science journey, Jupyter offers an intuitive platform for writing, testing, and sharing code. Its interactive interface enables data exploration, algorithm experimentation, and result visualization—all seamlessly integrated within a unified environment.

Key Features of Jupyter

Here are some key features and advantages of Jupyter:

1. **Interactive Computing:** Jupyter notebooks enable users to write and execute code interactively. This means you can run code cells individually and see the output immediately, fostering an iterative approach to coding and experimentation.
2. **Support for Multiple Languages:** While Jupyter was initially developed for Python (hence the name, which stands for Julia, Python, and R), it now supports various programming languages through its kernel system. This flexibility makes Jupyter suitable for various computational tasks and interdisciplinary collaboration.
3. **Rich Output:** Jupyter Notebooks support rich media integration, allowing users to generate interactive plots, charts, images, videos, and more directly within the document. This makes visualizing data, communicating findings, and creating compelling narratives easier.
4. **Integration with Data Science Libraries:** Jupyter seamlessly integrates with popular libraries and frameworks used in the data science ecosystem, such as NumPy, Pandas, Matplotlib, sci-kit-learn, TensorFlow, and PyTorch. This allows users to leverage the full power of these tools within the notebook environment for tasks like data manipulation, visualization, machine learning, and deep learning.

5. **Collaboration and Sharing:** Jupyter promotes collaboration and reproducibility by allowing users to share their notebooks with others via email, GitHub, or the Jupyter Notebook Viewer. This facilitates knowledge sharing, peer review, and interdisciplinary collaboration, as users can easily exchange ideas, code snippets, and best practices.

Jupyter in Data Science

Jupyter has become an indispensable tool for researchers, analysts, and developers in data science. Its seamless integration with popular libraries such as NumPy, pandas, and sci-kit-learn makes it the go-to choice for data manipulation, analysis, and machine learning. Jupyter provides a user-friendly interface, interactive capabilities, and robust collaboration features, making it an essential tool for anyone involved in data analysis, scientific research, education, or software development. Whether you're exploring data, building machine learning models, teaching a class, or conducting research, Jupyter empowers you to work more efficiently and share your insights with others.

Getting Started with Jupyter

Now that you can glimpse what Jupyter offers, it's time to dive in and experience its capabilities firsthand. Our Getting Started with Jupyter video will walk you through the basics of setting up and using Jupyter, empowering you to unleash the full potential of Python and embark on your data science journey with confidence.

So, let's jump into the world of Jupyter and unlock a world of possibilities in Python and data science!

1. Cheat Sheet: Python Basics

Package/Method	Description	Code Example
Comments	Comments are lines of text that are ignored by the Python interpreter when executing the code<./td>	<pre># This is a comment</pre>
Concatenation	Combines (concatenates) strings.	Syntax: <pre>5concatenated_string = string1 + string2</pre> Example: <pre>result = "Hello" + " John"</td></pre>
Data Types	- Integer - Float - Boolean - String	Example: <pre>x=7 # Integer Value y=12.4 # Float Value is_valid = True # Boolean Value is_valid = False # Boolean Value F_Name = "John" # String Value</pre>
Indexing	Accesses character at a specific index.	Example: <pre>my_string="Hello" char = my_string[0]</pre>

len()	Returns the length of a string.	Syntax: <code>len(string_name)</code> Example: <code>my_string="Hello"</code> <code>length = len(my_string)</code>
lower()	Converts string to lowercase.	Example: <code>my_string="Hello"</code> <code>uppercase_text =</code> <code>my_string.lower()</code>
print()	Prints the message or variable inside `()`.	Example: <code>print("Hello, world") print(a+b)</code>

Python Operators	- Addition (+): Adds two values together.- Subtraction (-): Subtracts one value from another.- Multiplication (*): Multiplies two values.- Division (/): Divides one value by another, returns a float.- Floor Division (//): Divides one value by another, returns the quotient as an integer.- Modulo (%): Returns the remainder after division.	Example: <pre> x = 9 y = 4 result_add= x + y # Addition result_sub= x - y # Subtraction result_mul= x * y # Multiplication result_div= x / y # Division result_fdiv= x // y # Floor Division result_mod= x % y # Modulo </pre>
replace()	Replaces substrings.	Example: <pre> my_string="Hello" new_text = my_string.replace("Hello", "Hi") </pre>
Slicing	Extracts a portion of the string.	Syntax: <pre> substring = string_name[start:end] </pre> Example: <pre> my_string="Hello" substring = my_string[0:5] </pre>

split()	Splits string into a list based on a delimiter.	Example: <pre>my_string="Hello" split_text = my_string.split(",")</pre>
strip()	Removes leading/trailing whitespace.	Example: <pre>my_string="Hello" trimmed = my_string.strip()</pre>
upper()	Converts string to uppercase.	Example: <pre>my_string="Hello" uppercase_text = my_string.upper()</pre>
Variable Assignment	Assigns a value to a variable.	Syntax: <pre>variable_name = value</pre> Example: <pre>name="John" # assigning John to variable name x = 5 # assigning 5 to variable x</pre>

II. Module 1 Summary: Python Basics

- Python can distinguish among data types such as integers, floats, strings, and Booleans.
- Integers are whole numbers that can be positive or negative.

- Floats are numbers that have decimal points; they can represent whole or fractional values.
- You can convert integers to floats using typecasting and vice-versa.
- You can convert integers and floats to strings.
- You can convert an integer or float to a Boolean: 0 becomes False, non-zero becomes True.
- Expressions in Python are a combination of values and operations used to produce a single result.
- Expressions perform mathematical operations such as addition, subtraction, multiplication, and so on.
- We can use // to perform integer division, which results in an integer value by discarding the fractional part.
- Python follows the order of operations (BODMAS) to perform operations with multiple expressions.
- Variables store and manipulate data, allowing you to access and modify values throughout your code.
- The assignment operator "=" assigns a value to a variable.
- Assigning another value to the same variable overrides the previous value of that variable.
- You can perform mathematical operations on variables using the same or different variables.
- Modifying the value of one variable will affect other variables only if they reference the same mutable object.
- Python string operations involve manipulating text data using tasks such as indexing, concatenation, slicing, and formatting.
- A string is usually written within double quotes or single quotes, including letters, white space, digits, or special characters.
- A string can be assigned to a variable and is an ordered sequence of characters.
- Characters in a string identify their index numbers, which can be positive or negative.
- Strings are sequences that support operations like indexing and slicing.
- You can input a stride value to perform slicing while operating on a string.

- Operations like concatenation and replication produce new strings, while finding the length of a string returns a number.
- You cannot modify an existing string; they are immutable.
- You can use escape sequences with a backslash (\) to change the layout of a string. (For example, \n for a new line, \t for a tab, and \\ for a backslash, etc.)
- In Python, you perform tasks such as searching, modifying, and formatting text data with its pre-built string methods.
- You apply a method to a string to change its value, resulting in another string.
- You can perform actions such as changing the case of characters in a string, replacing items in a string, finding items in a string, and so on using pre-built string methods.

III. Module 1 Glossary: Python Basics

Term	Definition
AI	AI (artificial intelligence) is the ability of a digital computer or computer-controlled robot to perform tasks commonly associated with intelligent beings.
Application development	Application development, or app development, is the process of planning, designing, creating, testing, and deploying a software application to perform various business operations.

Arithmetic Operations	Arithmetic operations are the basic calculations we make in everyday life like addition, subtraction, multiplication and division. It is also called as algebraic operations or mathematical operations.
Array of numbers	Set of numbers or objects that follow a pattern presented as an arrangement of rows and columns to explain multiplication.
Assignment operator in Python	Assignment operator is a type of Binary operator that helps in modifying the variable to its left with the use of its value to the right. The symbol used for assignment operator is "=".
Asterisk	Symbol "*" " used to perform various operations in Python.
Backslash	A backslash is an escape character used in Python strings to indicate that the character immediately following it should be treated in a special way, such as being treated as escaped character or raw string.
Boolean	Denoting a system of algebraic notation used to represent logical propositions by means of the binary digits 0 (false) and 1 (true).
Colon	A colon is used to represent an indented block. It is also used to fetch data and index ranges or arrays.
Concatenate	Link (things) together in a chain or series.
Data engineering	Data engineers are responsible for turning raw data into information that an organization can understand and use. Their work involves blending, testing, and optimizing data from numerous sources.

Data science	Data Science is an interdisciplinary field that focuses on extracting knowledge from data sets which are typically huge in amount. The field encompasses analysis, preparing data for analysis, and presenting findings to inform high-level decisions in an organization.
Data type	Data type refers to the type of value a variable has and what type of mathematical, relational or logical operations can be applied without causing an error.
Double quote	Symbol “ ” used to represent strings in Python.
Escape sequence	An escape sequence is two or more characters that often begin with an escape character that tell the computer to perform a function or command.
Expression	An expression is a combination of operators and operands that is interpreted to produce some other value.
Float	Python float () function is used to return a floating-point number from a number or a string representation of a numeric value.
Forward slash	Symbol “/” used to perform various operations in Python
Foundational	Denoting an underlying basis or principle; fundamental.

Immutable	Immutable Objects are of in-built datatypes like int, float, bool, string, Unicode, and tuple. In simple words, an immutable object can't be changed after it is created.
Integer	An integer is the number zero (0), a positive natural number (1, 2, 3, and so on) or a negative integer with a minus sign (-1, -2, -3, and so on.)
Manipulate	Is the process of modifying a string or creating a new string by making changes to existing strings.
Mathematical conventions	A mathematical convention is a fact, name, notation, or usage which is generally agreed upon by mathematicians.
Mathematical expressions	Expressions in math are mathematical statements that have a minimum of two terms containing numbers or variables, or both, connected by an operator in between.
Mathematical operations	The mathematical "operation" refers to calculating a value using operands and a math operator.
Negative indexing	Allows you to access elements of a sequence (such as a list, a string, or a tuple) from the end, using negative numbers as indexes.
Operands	The quantity on which an operation is to be done.
Operators in Python	Operators are used to perform operations on variables and values.

Parentheses	Parentheses is used to call an object.
Replicate	To make an exact copy of.
Sequence	A sequence is formally defined as a function whose domain is an interval of integers.
Single quote	Symbol ' ' used to represent strings in python.
Slicing in Python	Slicing is used to return a portion from defined list.
Special characters	A special character is one that is not considered a number or letter. Symbols, accent marks, and punctuation marks are considered special characters.
Stride value	Stride is the number of bytes from one row of pixels in memory to the next row of pixels in memory.
Strings	In Python, Strings are arrays of bytes representing Unicode characters.
Substring	A substring is a sequence of characters that are part of an original string.
Type casting	The process of converting one data type to another data type is called Typecasting or Type Coercion or Type Conversion.

Types in Python	Data types are the classification or categorization of data items. It represents the kind of value that tells what operations can be performed on a particular data.
Variables	Variables are containers for storing data values.

Module 2: Python Data Structures

I. Lists and Tuples

Cheat Sheet: Lists and Tuples

Use this cheat sheet for various packages/methods, their descriptions, and examples.

List

append()

The append() method is used to add an element to the end of a list.

Syntax:

```
1 list_name.append(element)
```

Example:

```
1 fruits = ["apple", "banana", "orange"]
2 fruits.append("mango") print(fruits)
```

copy()

The copy() method is used to create a shallow copy of a list.

Example:

```
1 my_list = [1, 2, 3, 4, 5]
2 new_list = my_list.copy() print(new_list)
3 # Output: [1, 2, 3, 4, 5]
```

count()

The count() method is used to count the number of occurrences of a specific element in a list in Python.

Example:

```
1 my_list = [1, 2, 2, 3, 4, 2, 5, 2]
2 count = my_list.count(2) print(count)
3 # Output: 4
```


Creating a List

A list is a built-in data type that represents an ordered and mutable collection of elements. Lists are enclosed in square brackets `[]` and elements are separated by commas.

Example:

```
1  fruits = ["apple", "banana", "orange", "mango"]
```

del

The `del` statement is used to remove an element from list. It removes the element at the specified index.

Example:

```
1  my_list = [10, 20, 30, 40, 50]
2  del my_list[2] # Removes the element at index 2 print(my_list)
3  # Output: [10, 20, 40, 50]
```

extend()

The `extend()` method is used to add multiple elements to a list. It takes an iterable (such as another list, tuple, or string) and appends each element of the iterable to the original list.

Syntax:

```
1  list_name.extend(iterable)
```

Example:

```
1  fruits = ["apple", "banana", "orange"]
2  more_fruits = ["mango", "grape"]
3  fruits.extend(more_fruits)
4  print(fruits)
```

Indexing

Indexing in a list allows you to access individual elements by their position. In Python, indexing starts from 0 for the first element and goes up to `'length_of_list - 1'`.

Example:

```
1 my_list = [10, 20, 30, 40, 50]
2 print(my_list[0])
3 # Output: 10 (accessing the first element)
4 print(my_list[-1])
5 # Output: 50 (accessing the last element using negative indexing)
```

insert()

The insert() method is used to insert an element.

Syntax:

```
1 list_name.insert(index, element)
```

Example:

```
1 my_list = [1, 2, 3, 4, 5]
2 my_list.insert(2, 6)
3 print(my_list)
```

Modifying a List

You can use indexing to modify or assign new values to specific elements in the list.

Example:

```
1 my_list = [10, 20, 30, 40, 50]
2 my_list[1] = 25 # Modifying the second element
3 print(my_list)
4 # Output: [10, 25, 30, 40, 50]
```

pop()

The pop() method is another way to remove an element from a list in Python. It removes and returns the element at the specified index. If you don't provide an index to the pop() method, it will remove and return the last element of the list by default

Example 1:

```
1 my_list = [10, 20, 30, 40, 50]
2 removed_element = my_list.pop(2) # Removes and returns the element at index 2
3 print(removed_element)
4 # Output: 30
5
6 print(my_list)
7 # Output: [10, 20, 40, 50]
```

Example 2:

```
1 my_list = [10, 20, 30, 40, 50]
2 removed_element = my_list.pop() # Removes and returns the last element
3 print(removed_element)
4 # Output: 50
5
6 print(my_list)
7 # Output: [10, 20, 30, 40]
```

remove()

To remove an element from a list. The `remove()` method removes the first occurrence of the specified value.

Example:

```
1 my_list = [10, 20, 30, 40, 50]
2 my_list.remove(30) # Removes the element 30
3 print(my_list)
4 # Output: [10, 20, 40, 50]
```

reverse()

The `reverse()` method is used to reverse the order of elements in a list

Example:

```
1 my_list = [1, 2, 3, 4, 5]
2 my_list.reverse() print(my_list)
3 # Output: [5, 4, 3, 2, 1]
```

Slicing

You can use slicing to access a range of elements from a list.

Syntax:

```
1 list_name[start:end:step]
```

Example:

```
1 my_list = [1, 2, 3, 4, 5]
2 print(my_list[1:4])
3 # Output: [2, 3, 4] (elements from index 1 to 3)
4
5 print(my_list[:3])
6 # Output: [1, 2, 3] (elements from the beginning up to index 2)
7
8 print(my_list[2:])
9 # Output: [3, 4, 5] (elements from index 2 to the end)
10
11 print(my_list[::2])
12 # Output: [1, 3, 5] (every second element)
```

sort()

The `sort()` method is used to sort the elements of a list in ascending order. If you want to sort the list in descending order, you can pass the `'reverse=True'` argument to the `sort()` method.

Example 1:

```
1 my_list = [5, 2, 8, 1, 9]
2 my_list.sort()
3 print(my_list)
4 # Output: [1, 2, 5, 8, 9]
```

Example 2:

```
1 my_list = [5, 2, 8, 1, 9]
2 my_list.sort(reverse=True)
3 print(my_list)
4 # Output: [9, 8, 5, 2, 1]
```

Tuple**count()**

The `count()` method for a tuple is used to count how many times a specified element appears in the tuple.

Syntax:

```
1 tuple.count(value)
```

Example:

```
1 fruits = ("apple", "banana", "apple", "orange")
2 print(fruits.count("apple")) #Counts the number of times apple is found in tuple.
3 #Output: 2
```

index()

The index() method in a tuple is used to find the first occurrence of a specified value and returns its position (index). If the value is not found, it raises a ValueError.

Syntax:

```
1 tuple.index(value)
```

Example:

```
1 fruits = ("apple", "banana", "orange", "apple")
2 print(fruits.index("apple")) #Returns the index value at which apple is present.
3 #Output: 0
```

sum()

The sum() function in Python can be used to calculate the sum of all elements in a tuple, provided that the elements are numeric (integers or floats).

Syntax:

```
1 sum(tuple)
```

Example:

```
1 numbers = (10, 20, 5, 30)
2 print(sum(numbers))
3 #Output: 65
```

min() and max()

Find the smallest (min()) or largest (max()) element in a tuple.

Example:

```
1 numbers = (10, 20, 5, 30)
2 print(min(numbers))
3 #Output: 5
4 print(max(numbers))
5 #Output: 30
```

len()

Get the number of elements in the tuple using len().

Syntax:

```
1 len(tuple)
```

Example:

```
1 fruits = ("apple", "banana", "orange")
2 print(len(fruits)) #Returns length of the tuple.
3 #Output: 3
```

II. Dictionaries & Sets

CheatSheet: Dictionaries & Sets

Dictionaries

Creating a Dictionary

A dictionary in Python is a built-in data structure that stores information in key-value pairs. Each key must be unique, and it is used to access its corresponding value.

Dictionaries are enclosed in curly braces {}, and each pair is separated by a comma.

Example:

```
1 dict_name = {} #Creates an empty dictionary
2 person = { "name": "John", "age": 30, "city": "New York"}
```

Accessing Values

You can access the values in a dictionary using their corresponding **keys**.

Syntax:

```
1 Value = dict_name["key_name"]
```

Example:

```
1 name = person["name"]
2 age = person["age"]
```

Add or modify

You can add or modify elements in a dictionary by assigning a value to a key using the assignment operator `=`. When you assign a value to a key that does not already exist, Python automatically creates a new key-value pair and adds it to the dictionary. However, if the key already exists, the existing value will be replaced with the new one.

Syntax:

```
1 dict_name[key] = value
```

Example:

```
1 person["Country"] = "USA" # A new entry will be created.
2 person["city"] = "Chicago" # Update the existing value for the same key
```

del

Removes the specified key-value pair from the dictionary. Raises a `KeyError` if the key does not exist.

Syntax:

```
1 del dict_name[key]
```

Example:

```
1 del person["Country"]
```

Key existence

You can check for the existence of a key in a dictionary using the `in` keyword.

Example:

```
1 if "name" in person:  
2     print("Name exists in the dictionary.")
```

Keys()

Retrieves all keys from the dictionary and converts them into a list. Useful for iterating or processing keys using list methods.

Syntax:

```
1 keys_list = list(dict_name.keys())
```

Example:

```
1 person_keys = list(person.keys())
```

Values()

Extracts all values from the dictionary and converts them into a list. This list can be used for further processing or analysis.

Syntax:

```
1 values_list = list(dict_name.values())
```

Example:

```
1 person_values = list(person.values())
```

Items()

Retrieves all key-value pairs as tuples and converts them into a list of tuples. Each tuple consists of a key and its corresponding value.

Syntax:

```
1 items_list = list(dict_name.items())
```

Example:

```
1 info = list(person.items())
```

Sets

Add()

Add elements to a set. You can use the `add()` method to insert a new element into a set. If the element already exists, it will not be added again, since sets automatically store only unique values.

Syntax:

```
1 set_name.add(element)
```

Example:

```
1 fruits.add("mango")
```

Defining Sets

A set in Python is an unordered collection that stores unique elements, meaning it automatically removes duplicates. Sets are enclosed in curly braces `{ }` and are commonly used when you need to store a group of distinct values or perform mathematical operations such as union, intersection, and difference.

Because sets are unordered, the elements have no fixed position or index, and their order may change each time you print or iterate over them.

Example: Note that these two sets will be used in the examples that follow.

```
1 empty_set = set() #Creating an Empty Set
2 fruits = {"apple", "banana", "orange"}
3 colors = ("orange", "red", "green")
```

in

Checks if an element exists in a set. Use the `in` keyword to verify whether a specific element is present in a set. It returns `True` if the element exists, and `False` otherwise.

Syntax:

```
1 element in set_name
```

Example:

```
1 "banana" in fruits
```

issubset()

Check if one set is a subset of another. The `issubset` method is used to determine whether all elements of one set exist within another set. If every element of the current set is found in the specified set, it returns `True`; otherwise, it returns `False`.

Syntax:

```
1 is_subset = set1.issubset(set2)
```

Example:

```
1 is_subset = fruits.issubset(colors)
```

issuperset()

Check if one set is a superset of another. The `issuperset()` method is used to verify whether the current set contains all elements of another set. It returns `True` if every element of the specified set exists within the current set; otherwise, it returns `False`.

Syntax:

```
1 is_superset = set1.issuperset(set2)
```

Example:

```
1 is_superset = colors.issuperset(fruits)
```

remove()

Remove a specific element from a set. The `remove()` method is used to delete a particular element from a set. If the element you try to remove does not exist, Python

will raise a `KeyError`, which means you must ensure the element is present before calling this method.

This method is useful when you are certain that the item you want to remove is already in the set.

Syntax:

```
1 set_name.remove(element)
```

Example:

```
1 fruits.remove("banana")
```

Set Operations

Perform various operations on sets: `union`, `intersection`, `difference`, `symmetric difference`.

Syntax:

```
1 union_set = set1.union(set2)
2 intersection_set = set1.intersection(set2)
3 difference_set = set1.difference(set2)
4 sym_diff_set = set1.symmetric_difference(set2)
```

Example:

```
1 combined = fruits.union(colors)
2 common = fruits.intersection(colors)
3 unique_to_fruits = fruits.difference(colors)
4 sym_diff = fruits.symmetric_difference(colors)
```

III. Module 2 Summary: Python Data Structures

- In Python, we often use tuples to group related data together. Tuples refer to ordered and immutable collections of elements.
- Tuples are usually written as comma-separated elements in parentheses "()".
- You can include strings, integers, and floats in tuples and access them using both positive and negative indices.
- You can perform operations such as combining, concatenating, and slicing on tuples.
- Tuples are immutable, so you need to create a new tuple to manipulate it.
- Tuples, termed nesting, can include other tuples of complex data types.
- You can access elements in a nested tuple through indexing.
- Lists in Python contain ordered collections of items that can hold elements of different types and are mutable, allowing for versatile data storage and manipulation.
- A list is an ordered sequence, represented with square brackets "[]".
- Lists possess mutability, rendering them akin to tuples.
- A list can contain strings, integers, and floats; you can nest lists within it.
- You can access each element in a list using both positive and negative indexing.
- Concatenating or appending a list will result in the modification of the same list.
- You can perform operations such as adding, deleting, splitting, and so forth on a list.
- You can separate elements in a list using delimiters.
- Aliasing occurs when multiple names refer to the same object.
- You can also clone a list to create another list.
- Dictionaries in Python are key-value pairs that provide a flexible way to store and retrieve data based on unique keys.
- Dictionaries consist of keys and values, both composed of string elements.
- You denote dictionaries using curly brackets {}.
- The keys necessitate immutability and uniqueness.
- The values may be either immutable or mutable, and they allow duplicates.
- You separate each key-value pair with a comma, and you can use color highlighting to make the key more visible.
- You can assign dictionaries to a variable.

- You use the key as an argument to retrieve the corresponding value.
- You can make additions and deletions to dictionaries.
- You can perform an operation on a dictionary to check the key, which results in a true or false output.
- You can apply methods to obtain a list of keys and values in a dictionary.
- Sets in Python are collections of unique elements, useful for tasks such as removing duplicates and performing set operations like union and intersection. Sets lack order.
- Curly brackets "{}" are helpful for defining elements of a set.
- Sets do not contain duplicate items.
- A list passed through the set function generates a set containing unique elements.
- You use "Set Operations" to perform actions such as adding, removing, and verifying elements in a set.
- You can combine sets using the ampersand "&" operator to obtain the common elements from both sets.
- You can use the Union function to combine two sets, including both the common and unique elements from both sets.
- The sub-set method is used to determine if two or more sets are subsets.

Module 3: Python Programming Fundamentals

I. II. Introduction to Loops

II. Conditions and Branching

Comparison operations

Comparison operations are essential in programming. They help compare values and make decisions based on the results.

Equality operator

The equality operator `==` checks if two values are equal. For example, in Python:

```
1 age = 25
2 if age == 25:
3     print("You are 25 years old.")
```

Inequality operator

The inequality operator `!=` checks if two values are not equal

```
1 if age != 30:
2     print("You are not 30 years old.")
```

Greater than and less than

You can also compare if one value is greater than another.

```
1 if age >= 20:
2     Print("Yes, the Age is greater than 20")
```

Branching

Branching is like making decisions in your program based on conditions. Think of it as real-life choices.

The IF statement

Consider a real-life scenario of entering a bar. If you're above a certain age, you can enter; otherwise, you cannot

```
1 age = 20
2 if age >= 21:
3     print("You can enter the bar.")
4 else:
5     print("Sorry, you cannot enter.")
```

The ELIF Statement

Sometimes, there are multiple conditions to check. For example, if you're not old enough for the bar, you can go to a movie instead.

```
1 if age >= 21:
2     print("You can enter the bar.")
3 elif age >= 18:
4     print("You can watch a movie.")
5 else:
6     print("Sorry, you cannot do either.")
```

Real-life example: Automated Teller Machine (ATM)

When a user interacts with an ATM, the software in the ATM can use branching to make decisions based on the user's input. For example, if the user selects "Withdraw Cash" the ATM can branch into different denominations of bills to dispense based on the amount requested.

```
1 user_choice = "Withdraw Cash"
2 if user_choice == "Withdraw Cash":
3     amount = int(input("Enter the amount to withdraw: "))
4     if amount % 10 == 0:
5         print("Amount dispensed: ", amount)
6     else:
7         print("Please enter a multiple of 10.")
8 else:
9     print("Thank you for using the ATM.")
```

Logical operators

Logical operators help combine and manipulate conditions.

The NOT operator

Real-life example: Notification settings

In a smartphone's notification settings, you can use the NOT operator to control when to send notifications. For example, you might only want to receive notifications when your phone is not in "Do Not Disturb" mode.

The not operator negates a condition.

```
1 is_do_not_disturb = True
2 if not is_do_not_disturb:
3     send_notification("New message received")
```

The AND operator

Real-life example: Access control

In a secure facility, you can use the AND operator to check multiple conditions for access. To open a high-security door, a person might need both a valid ID card and a matching fingerprint.

The AND operator checks if all required conditions are true, like needing both keys to open a safe.

```
1 has_valid_id_card = True
2 has_matching_fingerprint = True
3 if has_valid_id_card and has_matching_fingerprint:
4     open_high_security_door()
```

The OR operator

Real-life example: Movie night decision

When planning a movie night with friends, you can use the OR operator to decide on a movie genre. You'll choose a movie if at least one person is interested.

The OR operator checks if at least one condition is true. It's like choosing between different movies to watch.

```
1 friend1_likes_comedy = True
2 friend2_likes_action = False
3 friend3_likes_drama = False
4 if friend1_likes_comedy or friend2_likes_action or friend3_likes_drama:
5     choose a movie()
```

II. Exploring Python Functions

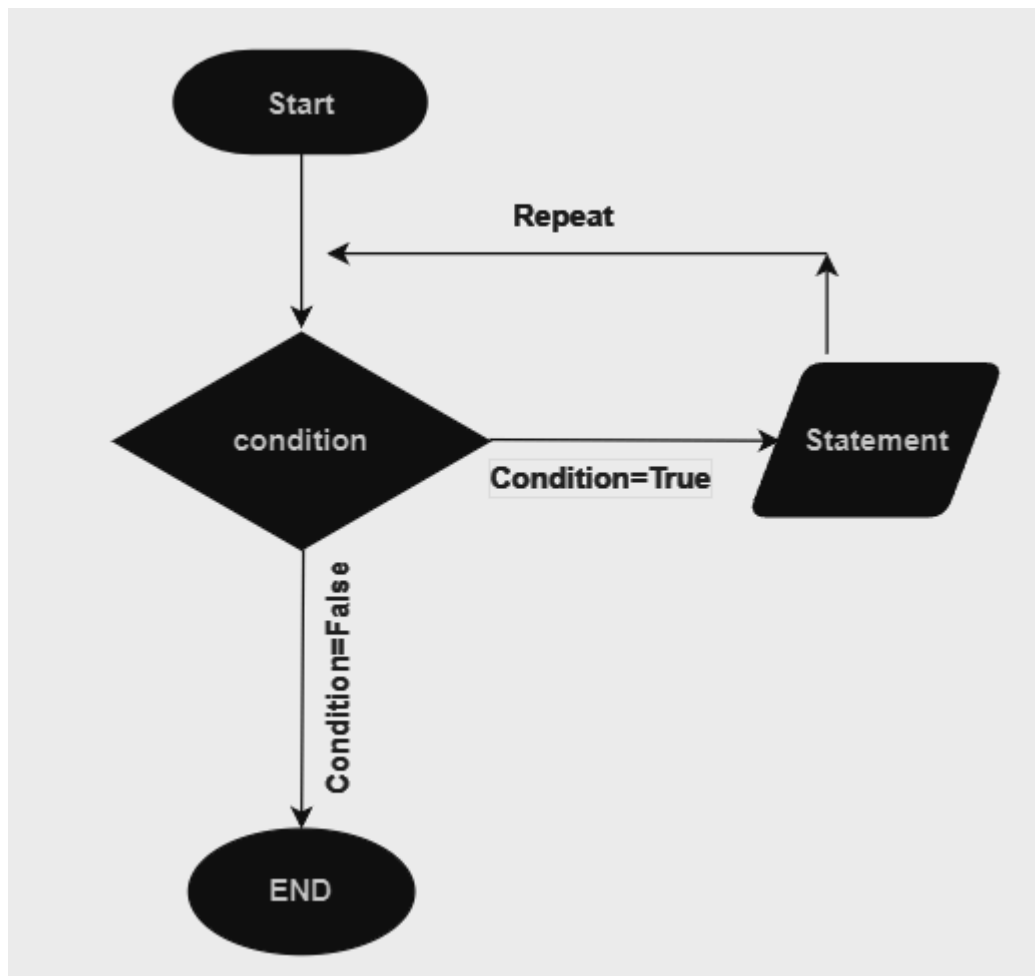
Introduction to Loops in Python

What is a Loop?

In programming, a loop is like a magic trick that allows a computer to do something over and over again. Imagine you are a magician's assistant, and your magician friend asks you to pull a rabbit out of a hat, but not just once - they want you to keep doing it until they tell you to stop. That is what loops do for computers - they repeat a set of instructions as many times as needed.

How Loop works?

Here's how it works in Python:



- **Start:** The for loop begins with the keyword for, followed by a variable that will take on each value in a sequence.
- **Condition:** After the variable, you specify the keyword in and a sequence, such as a list or a range, that the loop will iterate through.

If Condition True:

1. The loop takes the first value from the sequence and assigns it to the variable.
 2. The indented block of code following the loop header is executed using this value.
 3. The loop then moves to the next value in the sequence and repeats the process until all values have been used.
- **Statement:** Inside the indented block of the loop, you write the statements that you want to repeat for each value in the sequence.
 - **Repeat:** The loop continues to repeat the block of code for each value in the sequence until there are no more values left.

If Condition False:

1. Once all values in the sequence have been processed, the loop terminates automatically.
2. The loop completes its execution, and the program continues to the next statement after the loop.

The Need for Loops

Think about when you need to count from 1 to 10. Doing it manually is easy, but what if you had to count to a **million**? Typing all those numbers one by one would be a nightmare! This is where loops come in handy. They help computers repeat tasks quickly and accurately without getting tired.

Main Types of Loops

For Loops

For loops are like a superhero's checklist. A for loop in programming is a control structure that allows the repeated execution of a set of statements for each item in a sequence, such as elements in a list or numbers in a range, enabling efficient iteration and automation of tasks.

Syntax of for loop:

```
1  for val in sequence:
2      # statement(s) to be executed in sequence as a part of the loop.
```

Example of for loop:

Imagine you're a painter, and you want to paint a beautiful rainbow with seven colors. Instead of picking up each color one by one and painting the rainbow, you could tell a magical painter's assistant to do it for you. This is what a basic for loop does in programming.

We have a list of colors.

```
1 colors = ["red", "orange", "yellow", "green", "blue", "indigo", "violet"]
```

Let's print the colour name in the new line using for loop.

```
1 for color in colors:  
2     print(color)
```

In this example, the for loop picks each color from the colors list and prints it on the screen. You don't have to write the same code for each color - the loop does it automatically!

Sometimes you do not want to paint a rainbow, but *you want to count the number of steps to reach your goal*. A range-based for loop is like having a friendly step counter that helps you reach your target. Here is how you might use a for loop to count from 1 to 10:

```
2     print(number)
```

Here, the **range(1, 11)** generates a sequence from 1 to 10, and the for loop goes through each number in that sequence, printing it out. It's like taking 10 steps, and you're guided by the loop!

Range Function

The range function in Python generates an ordered sequence that can be used in loops. It takes one or two arguments:

- If given one argument (e.g., `range(11)`), it generates a sequence starting from 0 up to (but not including) the given number.

```
1 for number in range(11):  
2     print(number)
```

- If given two arguments (e.g., `range(1, 11)`), it generates a sequence starting from the first argument up to (but not including) the second argument.

```
1 for number in range(1, 11):
2     print(number)
```

The Enumerated For Loop

Have you ever needed to keep track of both the item and its position in a list? An enumerated for loop comes to your rescue. It's like having a personal assistant who not only hands you the item but also tells you where to find it.

Example:

```
1 fruits = ["apple", "banana", "orange"]
2 for index, fruit in enumerate(fruits):
3     print(f"At position {index}, I found a {fruit}")
```

With this loop, you not only get the fruit but also its position in the list. It's as if you have a magical guide pointing out each fruit's location!

While Loops

While loops are like a sleepless night at a friend's sleepover. Imagine you and your friends keep telling ghost stories until someone decides it's time to sleep. As long as no one says, "Let's sleep" you keep telling stories. A while loop works similarly - it repeats a task as long as a certain condition is true. It's like saying, "Hey computer, keep doing this until I say stop!"

Syntax:

```
1 while condition:
2     # Code to be executed while the condition is true
3     # Indentation is crucial to indicate the scope of the loop
```

Example:

Here's how you might use a while loop to count from 1 to 10:

```
1 count = 1
2 while count <= 10:
3     print(count)
4     count += 1
```

Here's a breakdown of the above code:

1. There is a variable named **count** initialized with the value 1.
2. The while loop is used to repeatedly execute a block of code as long as a given condition is True. In this case, the condition is **count <= 10**, meaning the loop will continue as long as count is less than or equal to 10.
3. Inside the loop:
 - The **print(count)** statement outputs the current value of the count variable.
 - The **count += 1** statement increments the value of count by 1. This step ensures that the loop will eventually terminate when count becomes greater than 10.
4. The loop will continue executing as long as the condition count <= 10 is satisfied.
5. The loop will print the numbers 1 to 10 in consecutive order since the print statement is inside the loop block and executed during each iteration.
6. Once count reaches 11, the condition count <= 10 will evaluate to False, and the loop will terminate.
7. The output of the code will be the numbers 1 to 10, each printed on a separate line.

The Loop Flow

Both for and while loops have their special moves, but they follow a pattern:

- **Initialization:** You set up things like a starting point or conditions.
- **Condition:** You decide when the loop should keep going and when it should stop.
- **Execution:** You do the task inside the loop.
- **Update:** You make changes to your starting point or conditions to move forward.
- **Repeat:** The loop goes back to step 2 until the condition is no longer true.

When to Use Each

For Loops: Use for loops when you know the number of iterations in advance and want to process each element in a sequence. They are best suited for iterating over collections and sequences where the length is known.

While Loops: Use while loops when you need to perform a task repeatedly as long as a certain condition holds true. While loops are particularly useful for situations where the number of iterations is uncertain or where you're waiting for a specific condition to be met.

Summary

In this adventure into coding, we explored loops in Python - special tools that help us do things over and over again without getting tired. We met two types of loops: **"for loops"** and **"while loops."**

For Loops were like helpers that made us repeat tasks in order. We painted colors, counted numbers, and even got a helper to tell us where things were in a list. For loops made our job easier and made our code look cleaner.

While Loops were like detectives that kept doing something as long as a rule was true. They helped us take steps, guess numbers, and work until we were tired. While loops were like smart assistants that didn't stop until we said so.

Exploring Python Functions

Introduction to functions

A function is a fundamental building block that encapsulates specific actions or computations. As in mathematics, where functions take inputs and produce outputs, programming functions perform similarly. They take inputs, execute predefined actions or calculations, and then return an output.

Purpose of functions

Functions promote code modularity and reusability. Imagine you have a task that needs to be performed multiple times within a program. Instead of duplicating the same code at various places, you can define a function once and call it whenever you need that task. This reduces redundancy and makes the code easier to manage and maintain.

Benefits of using functions

- **Modularity:** Functions break down complex tasks into manageable components
- **Reusability:** Functions can be used multiple times without rewriting code
- **Readability:** Functions with meaningful names enhance code understanding
- **Debugging:** Isolating functions eases troubleshooting and issue fixing
- **Abstraction:** Functions simplify complex processes behind a user-friendly interface
- **Collaboration:** Team members can work on different functions concurrently
- **Maintenance:** Changes made in a function automatically apply wherever it's used

How functions take inputs, perform tasks, and produce outputs

Inputs (Parameters)

Functions operate on data, and they can receive data as input. These inputs are known as *parameters* or *arguments*. Parameters provide functions with the necessary information they need to perform their tasks. Consider parameters as values you pass to a function, allowing it to work with specific data.

Performing tasks

Once a function receives its input (parameters), it executes predefined actions or computations. These actions can include calculations, operations on data, or even more complex tasks. The purpose of a function determines the tasks it performs. For

instance, a function could calculate the sum of numbers, sort a list, format text, or fetch data from a database.

Producing outputs

After performing its tasks, a function can produce an output. This output is the result of the operations carried out within the function. It's the value that the function “returns” to the code that called it. Think of the output as the end product of the function's work. You can use this output in your code, assign it to variables, pass it to other functions, or even print it out for display.

Example:

Consider a function named `calculate_total` that takes two numbers as input (parameters), adds them together, and then produces the sum as the output. Here's how it works:

```
1 def calculate_total(a, b): # Parameters: a and b
2     total = a + b          # Task: Addition
3     return total           # Output: Sum of a and b
4
5 result = calculate_total(5, 7) # Calling the function with inputs 5 and 7
6 print(result) # Output: 12
```

Python's built-in functions

Python has a rich set of built-in functions that provide a wide range of functionalities. These functions are readily available for you to use, and you don't need to be concerned about how they are implemented internally. Instead, you can focus on understanding what each function does and how to use it effectively.

Using built-in functions or Pre-defined functions

To use a built-in function, you simply call the function's name followed by parentheses. Any required arguments or parameters are passed into the function within these parentheses. The function then performs its predefined task and may return an output you can use in your code.

Here are a few examples of commonly used built-in functions:

- **len():** Calculates the length of a sequence or collection

```
1 string_length = len("Hello, World!") # Output: 13
2 list_length = len([1, 2, 3, 4, 5]) # Output: 5
```

- **sum():** Adds up the elements in an iterable (list, tuple, and so on)

```
1 total = sum([10, 20, 30, 40, 50]) # Output: 150
```

- **max():** Returns the maximum value in an iterable

```
1 highest = max([5, 12, 8, 23, 16]) # Output: 23
```

- **min():** Returns the minimum value in an iterable

```
1 lowest = min([5, 12, 8, 23, 16]) # Output: 5
```

Python's built-in functions offer a wide array of functionalities, from basic operations like `len()` and `sum()` to more specialized tasks.

Defining your functions

Defining a function is like creating your mini-program:

Use `def` followed by the function name and parentheses. Here is the syntax to define a function:

```
1 def function_name():
2     pass
```

A `"pass"` statement in a programming function is a placeholder or a no-op (no operation) statement. Use it when you want to define a function or a code block syntactically but do not want to specify any functionality or implementation at that moment.

- **Placeholder:** `"pass"` acts as a temporary placeholder for future code that you intend to write within a function or a code block.
- **Syntax Requirement:** In many programming languages like Python, using `"pass"` is necessary when you define a function or a conditional block. It

ensures that the code remains syntactically correct, even if it doesn't do anything yet.

- **No Operation:** `"pass"` itself doesn't perform any meaningful action. When the interpreter encounters `"pass"`, it simply moves on to the next statement without executing any code.

Function Parameters:

- Parameters are like inputs for functions
- They go inside parentheses when defining the function
- Functions can have multiple parameters

Example:

```
1 def greet(name):
2     return "Hello, " + name
3
4 result = greet("Alice")
5 print(result) # Output: Hello, Alice
```

Docstrings (Documentation Strings):

- Docstrings explain what a function does
- Placed inside triple quotes under the function definition
- Helps other developers understand your function

Example:

```
1 def multiply(a, b):
2     """
3     This function multiplies two numbers.
4     Input: a (number), b (number)
5     Output: Product of a and b
6     """
7     print(a * b)
8 multiply(2,6)
```

Return Statement:

- Return gives back a value from a function
- Ends the function's execution and sends the result

- A function can return various types of data

Example:

```
1 def add(a, b):  
2     return a + b  
3  
4 sum_result = add(3, 5) # sum_result gets the value 8
```

Understanding scopes and variables

Scope is where a variable can be seen and used:

- **Global Scope:** Variables defined outside functions; accessible everywhere
- **Local Scope:** Variables inside functions; only usable within that function

Part 1: Global variable declaration

```
1 global_variable = "I'm global"
```

This line initializes a global variable called `global_variable` and assigns it the value "I'm global".

Global variables are accessible throughout the entire program, both inside and outside functions.

Part 2: Function definition

```
1 def example_function():  
2     local_variable = "I'm local"  
3     print(global_variable) # Accessing global variable  
4     print(local_variable)  # Accessing local variable
```

Here, you define a function called `example_function()`.

Within this function:

- A local variable named `local_variable` is declared and initialized with the string value "I'm local." This variable is local to the function and can only be accessed within the function's scope.

- The function then prints the values of both the **global variable (global_variable)** and the **local variable (local_variable)**. It demonstrates that you can access global and local variables within a function.

Part 3: Function call

```
1 example_function()
```

In this part, you call the `example_function()` by invoking it. This results in the function's code being executed.

As a result of this function call, it will print the values of the global and local variables within the function.

Part 4: Accessing global variable outside the function

```
1 print(global_variable) # Accessible outside the function
```

Here, you use the `add_element` function to add three elements (42, 17, and 99) to the `my_list`. These are added one at a time using function calls.

Part 5: Print the current list

```
1 # Print the current list
2 print("Current list:", my_list)
```

This part simply prints the current state of the `my_list` to the console, allowing us to see the elements that have been added so far.

Part 6: Remove elements from the list

```
1 # Remove an element from the list using the remove_element function
2 remove_element(my_list, 17)
3 remove_element(my_list, 55) # This will print a message since 55 is not in the list
```

In this part, you use the `remove_element` function to remove elements from the `my_list`. First, you attempt to remove 17 (which is in the list), and then you try to remove 55 (which is not in the list). **The second call to `remove_element` will print a message indicating that 55 was not found.**

Part 7: Print the updated list

```
1 # Print the updated list
2 print("Updated list:", my_list)
```

Finally, you print the updated `my_list` to the console. This allows us to observe the modifications made to the list by adding and removing elements using the defined functions.

Conclusion

Congratulations! You've completed the Reading Instruction Lab on Python functions. You've gained a solid understanding of functions, their significance, and how to create and use them effectively. These skills will empower you to write more organized, modular, and powerful code in your Python projects.

Exception Handling

What are exceptions?

Exceptions are alerts when something unexpected happens while running a program. It could be a mistake in the code or a situation that was not planned for. Python can raise these alerts automatically, but we can also trigger them on purpose using the `raise` command. The cool part is that we can prevent our program from crashing by handling exceptions.

Errors vs. Exceptions

Hold on, what is the difference between errors and exceptions? Well, **errors** are usually big problems that come from the computer or the system. They often make the program stop working completely. On the other hand, **exceptions** are more like issues we can control. They happen because of something we did in our code and can usually be fixed, so the program keeps going.

Here is the difference between **Errors** and **exceptions**:

Aspect	Errors	Exceptions
Origin	Errors are typically caused by the environment, hardware, or operating system.	Exceptions are usually a result of problematic code execution within the program.
Nature	Errors are often severe and can lead to program crashes or abnormal termination.	Exceptions are generally less severe and can be caught and handled to prevent program termination.
Handling	Errors are not usually caught or handled by the program itself.	Exceptions can be caught using try-except blocks and dealt with gracefully, allowing the program to continue execution.
Examples	Examples include "SyntaxError" due to incorrect syntax or "NameError" when a variable is not defined.	Examples include "ZeroDivisionError" when dividing by zero, or "FileNotFoundError" when attempting to open a non-existent file.
Categorization	Errors are not classified into categories.	Exceptions are categorized into various classes, such as "ArithmeticError," "IOError," "ValueError," etc., based on their nature.

Common Exceptions in Python

Here are a few examples of exceptions we often run into and can handle using this tool:

ZeroDivisionError

This error arises when an attempt is made to divide a number by zero. Division by zero is undefined in mathematics, causing an arithmetic error.

For example:

```
1 result = 10 / 0
2 print(result)
3 # Raises ZeroDivisionError
```

ValueError

This error occurs when an inappropriate value is used within the code. An example of this is when trying to convert a non-numeric string to an integer.

For example:

```
1 num = int("abc")
2 # Raises ValueError
```

FileNotFoundError

This exception is encountered when an attempt is made to access a file that does not exist.

For example:

```
1 with open("nonexistent_file.txt", "r") as file:
2     content = file.read() # Raises FileNotFoundError
```

IndexError

An IndexError occurs when an index is used to access an element in a list that is outside the valid index range.

For example:

```
1 my_list = [1, 2, 3]
2 value = my_list[1] # No IndexError, within range
3 missing = my_list[5] # Raises IndexError
```

KeyError

The `KeyError` arises when an attempt is made to access a non-existent key in a dictionary.

For example:

```
1 my_dict = {"name": "Alice", "age": 30}
2 value = my_dict.get("city") # No KeyError, using .get() method
3 missing = my_dict["city"] # Raises KeyError
```

TypeError

The `TypeError` occurs when an object is used in an incompatible manner. An example includes trying to concatenate a string and an integer.

For example:

```
1 result = "hello" + 5
2 # Raises TypeError
```

AttributeError

An `AttributeError` occurs when an attribute or method is accessed on an object that doesn't possess that specific attribute or method.

For example:

```
1 text = "example"
2 length = len(text) # No AttributeError, correct method usage
3 missing = text.some_method() # Raises AttributeError
```

ImportError

This error is encountered when an attempt is made to import a module that is unavailable.

For example: `import non_existent_module`

Note: Please remember, the exceptions you will encounter are not limited to just these. There are many more in Python. However, there is no need to worry. By using the technique provided below and following the correct syntax, you will be able to handle any exceptions that come your way.

Handling Exceptions

Python has a handy tool called `try and except` that helps us manage exceptions.

Try and Except: You can use the try and except blocks to prevent your program from crashing due to exceptions.

Here's how they work:

1. The code that may result in an exception is contained in the try block.
2. If an exception occurs, the code directly jumps to except block.
3. In the except block, you can define how to handle the exception gracefully, like displaying an error message or taking alternative actions.
4. After the except block, the program continues executing the remaining code.

Example: Attempting to divide by zero

```
1  # using Try- except
2  try:
3      # Attempting to divide 10 by 0
4      result = 10 / 0
5  except ZeroDivisionError:
6      # Handling the ZeroDivisionError and printing an error message
7      print("Error: Cannot divide by zero")
8  # This line will be executed regardless of whether an exception occurred
9  print("outside of try and except block")
```

Objects and Classes

Introduction to classes and object

Python is an object-oriented programming (OOP) language that uses a paradigm centered around objects and classes.

Let's look at these fundamental concepts.

Classes

A class is a blueprint or template for creating objects. It defines the structure and behavior that its objects will have.

Think of a class as a cookie cutter and objects as the cookies cut from that template.

In Python, you can create classes using the `class` keyword.

Creating Classes

When you create a class, you specify the `attributes`(data) and `methods` (functions) that objects of that class will have.

`Attributes` are defined as variables within the class, and `methods` are defined as functions.

For example, you can design a "Car" class with attributes such as "color" and "speed," along with methods like "accelerate."

Objects

An *object* is a fundamental unit in Python that represents a real-world entity or concept. Objects can be tangible (like a car) or abstract (like a student's grade).

Every object has two main characteristics:

State

The *attributes or data* that describe the object. For your "Car" object, this might include attributes like "color", "speed", and "fuel level".

Behavior

The *actions or methods* that the object can perform. In Python, methods are functions that belong to objects and can change the object's state or perform specific operations.

Instantiating objects

- Once you've defined a class, you can create individual objects (instances) based on that class.
- Each object is independent and has its own set of attributes and methods.
- To create an object, you use the class name followed by parentheses, so:
`"my_car = Car()"`

Interacting with objects

You interact with objects by calling their methods or accessing their attributes using dot notation.

For example, if you have a Car object named **my_car**, you can set its color with **my_car.color = "blue"** and accelerate it with **my_car.accelerate()** if there's an accelerate method defined in the class.

Structure of classes and object code

Please don't directly copy and use this code because it is a template for explanation and not for specific results.

Class declaration (class ClassName)

- The `class` keyword is used to declare a class in Python.
- `ClassName` is the name of the class, typically following CamelCase naming conventions.

```
1 class ClassName:
```

Class attributes (class_attribute = value)

- Class attributes are variables shared among all class instances (objects).
- They are defined within the class but outside of any methods.

```
1 class ClassName:
2     # Class attributes (shared by all instances)
3     class_attribute = value
```

Constructor method (def `__init__(self, attribute1, attribute2, ...):`)

- The `__init__` method is a special method known as the constructor.
- It initializes the **instance attributes** (also called instance variables) when an object is created.
- The `self` parameter is the first parameter of the constructor, referring to the instance being created.
- **attribute1**, **attribute2**, and so on are parameters passed to the constructor when creating an object.
- Inside the constructor, `self.attribute1`, `self.attribute2`, and so on are used to assign values to instance attributes.

```
1 class ClassName:
2     # Class attributes (shared by all instances)
3     class_attribute = value
4
5     # Constructor method (initialize instance attributes)
6     def __init__(self, attribute1, attribute2, ...):
7         pass
8         # ...
```

Instance attributes (self.attribute1 = attribute1)

- Instance attributes are variables that store data specific to each class instance.
- They are initialized within the `__init__` method using the `self` keyword followed by the attribute name.
- These attributes hold unique data for each object created from the class.

```

1  class ClassName:
2      # Class attributes (shared by all instances)
3      class_attribute = value
4
5      # Constructor method (initialize instance attributes)
6      def __init__(self, attribute1, attribute2, ...):
7          self.attribute1 = attribute1
8          self.attribute2 = attribute2
9          # ...

```

Instance methods (def method1(self, parameter1, parameter2, ...):)

- Instance methods are functions defined within the class.
- They operate on the instance's data (instance attributes) and can perform actions specific to instances.
- The **self** parameter is required in instance methods, allowing them to access instance attributes and call other methods within the class.

```

1  class ClassName:
2      # Class attributes (shared by all instances)
3      class_attribute = value
4
5      # Constructor method (initialize instance attributes)
6      def __init__(self, attribute1, attribute2, ...):
7          self.attribute1 = attribute1
8          self.attribute2 = attribute2
9          # ...
10
11     # Instance methods (functions)
12     def method1(self, parameter1, parameter2, ...):
13         # Method logic
14         pass

```

Using the same steps you can define multiple instance methods.

```

1  class ClassName:
2      # Class attributes (shared by all instances)
3      class_attribute = value
4
5      # Constructor method (initialize instance attributes)
6      def __init__(self, attribute1, attribute2, ...):
7          self.attribute1 = attribute1
8          self.attribute2 = attribute2
9          # ...
10
11     # Instance methods (functions)
12     def method1(self, parameter1, parameter2, ...):
13         # Method logic
14         pass
15
16     def method2(self, parameter1, parameter2, ...):
17         # Method logic
18         pass

```

Note: Now, you have successfully created a dummy class.

Creating objects (Instances)

- To create objects (instances) of the class, you call the class like a function and provide arguments the constructor requires.
- Each object is a distinct instance of the class, with its own instance attributes and the ability to call methods defined in the class.

```

1  # Create objects (instances) of the class
2  object1 = ClassName(arg1, arg2, ...)
3  object2 = ClassName(arg1, arg2, ...)

```

Calling methods on objects

- In this section, you will call methods on objects, specifically `object1` and `object2`.
- The methods `method1` and `method2` are defined in the `ClassName` class, and you're calling them on `object1` and `object2` respectively.
- You pass values `param1_value` and `param2_value` as arguments to these methods. These arguments are used within the method's logic.

Method 1: Using dot notation

- This is the most straightforward way to call an object's method. In this, use the dot notation (**object.method()**) to invoke the method on the object directly.
- For example, `result1 = object1.method1(param1_value, param2_value, ...)` calls method1 on object1.

```
1 # Calling methods on objects
2 # Method 1: Using dot notation
3 result1 = object1.method1(param1_value, param2_value, ...)
4 result2 = object2.method2(param1_value, param2_value, ...)
```

Method 2: Assigning object methods to variables

- Here's an alternative way to call an object's method by assigning the method reference to a variable.
- `method_reference = object1.method1` assigns the method **method1** of **object1** to the variable **method_reference**.
- Later, call the method using the variable like this: **result3 = method_reference(param1_value, param2_value, ...)**.

```
1 # Method 2: Assigning object methods to variables
2 method_reference = object1.method1 # Assign the method to a variable
3 result3 = method_reference(param1_value, param2_value, ...)
```

Accessing object attributes

- Here, you are accessing an object's attribute using dot notation.
- `attribute_value = object1.attribute1` retrieves the value of the attribute **attribute1** from **object1** and assigns it to the variable **attribute_value**.

```
1 # Accessing object attributes
2 attribute_value = object1.attribute1 # Access the attribute using dot notation
```

Modifying object attributes

- You will modify an object's attribute using dot notation.

- `object1.attribute2 = new_value` sets the attribute **attribute2** of **object1** to the new value **new_value**.

```
1 # Modifying object attributes
2 object1.attribute2 = new_value # Change the value of an attribute using dot notation
```

Accessing class attributes (shared by all instances)

- Finally, access a class attribute shared by all class instances.
- `class_attr_value = ClassName.class_attribute` accesses the class attribute `class_attribute` from the `ClassName` `class` and assigns its value to the variable `class_attr_value`.

```
1 # Accessing class attributes (shared by all instances)
2 class_attr_value = ClassName.class_attribute
```

Real-world example

Let's write a python program that simulates a simple car class, allowing you to create car instances, accelerate them, and display their current speeds.

Let's start by defining a `Car` class that includes the following attributes and methods:

- Class attribute `max_speed`, which is set to **120 km/h**.
- Constructor method `__init__` that takes parameters for the **car's make, model, color, and an optional speed (defaulting to 0)**. This method initializes instance attributes for make, model, color, and speed.
- Method `accelerate(self, acceleration)` that allows the car to accelerate. If the acceleration does not exceed the `max_speed`, update the **car's speed** attribute. Otherwise, set the speed to the `max_speed`.
- Method `get_speed(self)` that returns the current speed of the car.

```

1  class Car:
2      # Class attribute (shared by all instances)
3      max_speed = 120 # Maximum speed in km/h
4
5      # Constructor method (initialize instance attributes)
6      def __init__(self, make, model, color, speed=0):
7          self.make = make
8          self.model = model
9          self.color = color
10         self.speed = speed # Initial speed is set to 0
11
12         # Method for accelerating the car
13         def accelerate(self, acceleration):
14             if self.speed + acceleration <= Car.max_speed:
15                 self.speed += acceleration
16             else:
17                 self.speed = Car.max_speed
18
19         # Method to get the current speed of the car
20         def get_speed(self):
21             return self.speed

```

Now, you will instantiate two objects of the `Car` class, each with the following characteristics:

- car1: **Make = "Toyota", Model = "Camry", Color = "Blue"**
- car2: **Make = "Honda", Model = "Civic", Color = "Red"**

```

1  # Create objects (instances) of the Car class
2  car1 = Car("Toyota", "Camry", "Blue")
3  car2 = Car("Honda", "Civic", "Red")

```

Using the `accelerate` method, you will increase the speed of car1 by 30 km/h and car2 by 20 km/h.

```

1  # Accelerate the cars
2  car1.accelerate(30)
3  car2.accelerate(20)

```

Lastly, you will display the current speed of each car by utilizing the `get_speed` method.

```

1  # Print the current speeds of the cars
2  print(f"{car1.make} {car1.model} is currently at {car1.get_speed()} km/h.")
3  print(f"{car2.make} {car2.model} is currently at {car2.get_speed()} km/h.")

```

Module 3 Summary: Python Programming Fundamentals

- Python conditions use "if" statements to execute code based on true/false conditions created by comparisons and Boolean expressions.
- Comparison operations require using comparison operators such as == (equal to), > (greater than), and < (less than).
- Python uses the "!=" operator to determine whether two values are not equal.
- You can compare integers, strings, and floats.
- Python branching directs program flow by using conditional statements (for example, if, else, elif) to execute different code blocks based on conditions or tests.
- You can use the "if" statement with conditions to define actions if true.
- To perform actions when all previous conditions are false, you can use the "else" statement without a condition.
- The elif statement allows for additional checks only if the initial condition is false.
- To execute various operations on Boolean values, we use Boolean logic operators.
- Python loops are control structures that automate repetitive tasks and iterate over data structures like lists or dictionaries.
- The range() function generates a sequence of numbers with a specified start, stop, and step value for loops in Python.
- A for loop in Python iterates over a sequence, such as a list, tuple, or string, and executes a block of code for each item in the sequence.
- A while loop in Python executes a block of code as long as a specified condition remains true.
- Python functions are reusable code blocks that perform specific tasks, take input parameters, and often return results, enhancing code modularity and reusability.

- You may or may not have written the codes that are often included in functions.
- Python has a set of built-in functions such as "len" to find the length of a sequence or "sum" to find the total sum of a sequence.
- The "sorted" function creates a new sorted list, while "sort" sorts items in the original list.
- You can also create your own functions in Python.
- To ensure clarity and organization and facilitate understanding and maintenance of the code, developers must document functions using a documentation string enclosed in three quotes.
- The help command will return the documentation defined for a particular function.
- A function can have multiple parameters.
- If a function does not include a return statement, it returns None by default.
- You can use the "pass" keyword in a function to indicate that it does nothing (a placeholder for future code).
- A function will usually perform more than one task.
- In Python, the scope of a variable determines where you can access or modify that variable. Global scope allows access from anywhere, while local scope restricts it to a block or function.
- In Python, a programmer defines a local variable within a specific block or function, which can only be accessed or modified within that block or function.
- In Python, a global variable is a variable defined at the top level of a program that any part of the code can access or modify.
- III. Exploring Python Functions
- in Python is a mechanism for managing and responding to errors and exceptions that may occur during program execution, preventing them from crashing the program.
- In Python, you use the "try-except" statement to attempt a block of code and specify alternative actions to execute if an error occurs, allowing you to handle exceptions.
- In Python, you use the "try-except-else" statement to attempt a block of code, handle exceptions in the "except" block, and execute code in the "else" block when no exceptions occur.

- Python developers use the "try-except-else-finally" statement to attempt a block of code, catch exceptions in the "except" block, execute code in the "else" block when no exceptions occur, and ensure that the "finally" block always runs, regardless of whether exception was raised or not.
- In Python, objects are instances of classes that encapsulate data and behavior, serving as the foundation for creating and working with various data types and custom data structures.
- To determine the type of an object in Python, you can use the ``type()`` command.
- Methods may modify an object's internal state, but the object's type usually remains the same.
- Classes in Python are blueprints for creating objects, defining their attributes and methods, enabling code organization, and object-oriented programming.
- Function "init" is a special method used to initialize data attributes.
- We can create instances of a class in Python.
- Data attributes consist of the data defining the objects.
- Methods are functions that interact and change the data attributes.
- The method has a function that requires the self as well as other parameters.

Cheat Sheet: Python Programming Fundamentals

Use this cheat sheet for various packages/methods, their descriptions, and examples.

AND

Returns **True** if both statement1 and statement2 are True. Otherwise, returns **False**.

Example:

```
1 statement1 and statement2
```

Syntax:

```
1 marks = 90
2 attendance_percentage = 87
3
4 if marks >= 80 and attendance_percentage >= 85:
5     print("qualify for honors")
6 else:
7     print("Not qualified for honors")
8
9 # Output = qualify for honors
```

Class Definition

Defines a blueprint for creating objects and defining their attributes and behaviors.

Syntax:

```
1 class ClassName: # Class attributes and methods
```

Example:

```
1 class Person:
2     def __init__(self, name, age):
3         self.name = name
4         self.age = age
```

Define Function

A function is a reusable block of code that performs a specific task or set of tasks when called.

Syntax:

```
1 def function_name(parameters): # Function body
```

Example:

```
1 def greet(name): print("Hello,", name)
```

Equal (==)

Checks if two values are equal.

Syntax:

```
1 variable1 == variable2
```

Example 1:

```
1 5 == 5
```

returns **True**

Example 2:

```
1 age = 25
2 age == 30
```

returns **False**

For Loop

A **for** loop repeatedly executes a block of code for a specified number of iterations or over a sequence of elements (list, range, string, etc.).

Syntax:

```
1 for variable in sequence: # Code to repeat
```

Example 1:

```
1 for num in range(1, 10):  
2     print(num)
```

Example 2:

```
1 fruits = ["apple", "banana", "orange", "grape", "kiwi"]  
2 for fruit in fruits:  
3     print(fruit)
```

Function Call

A function call is the act of executing the code within the function using the provided arguments.

Syntax:

```
1 function_name(arguments)
```

Example:

```
1 greet("Alice")
```

Greater Than or Equal To(>=)

Checks if the value of variable1 is greater than or equal to variable2.

Syntax:

```
1 variable1 >= variable2
```

Example 1:

```
1 5 >= 5 and 9 >= 5
```

returns **True**

Example 2:

```
1 quantity = 105
2 minimum = 100
3 quantity >= minimum
```

returns **True**

Greater Than(>)

Checks if the value of variable1 is greater than variable2.

Syntax:

```
1 variable1 > variable2
```

Example 1:

```
1 9 > 6
```

returns **True**

Example 2:

```
1 age = 20
2 max_age = 25
3 age > max_age
```

returns **False**

If Statement

Executes code block `if` the condition is `True`.

Syntax:

```
1  if condition: #code block for if statement
```

Example:

```
1  if temperature > 30:  
2  |    print("It's a hot day!")
```

If-Elif-Else

Executes the first code block if condition1 is `True`, otherwise checks condition2, and so on. If no condition is `True`, the else block is executed.

Syntax:

```
1  if condition1:  
2  |    # Code if condition1 is True  
3  
4  elif condition2:  
5  |    # Code if condition2 is True  
6  
7  else:  
8  |    # Code if no condition is True
```

Example:

```
1  score = 85 # Example score  
2  if score >= 90:  
3  |    print("You got an A!")  
4  elif score >= 80:  
5  |    print("You got a B.")  
6  else:  
7  |    print("You need to work harder.")  
8  
9  # Output = You got a B.
```

If-Else Statement

Executes the first code block if the condition is **True**, otherwise the second block.

Syntax:

```
1 if condition: # Code, if condition is True
2 else: # Code, if condition is False
```

Example:

```
1 if age >= 18:
2     print("You're an adult.")
3 else:
4     print("You're not an adult yet.")
```

Less Than or Equal To(<=)

Checks if the value of variable1 is less than or equal to variable2.

Syntax:

```
1 variable1 <= variable2
```

Example 1:

```
1 5 <= 5 and 3 <= 5
```

returns **True**

Example 2:

```
1 size = 38
2 max_size = 40
3 size <= max_size
```

returns **True**

Less Than(<)

Checks if the value of variable1 is less than variable2.

Syntax:

```
1 variable1 < variable2
```

Example 1:

```
1 4 < 6
```

returns **True**

Example 2:

```
1 score = 60
2 passing_score = 65
3 score < passing_score
```

returns **True**

Loop Controls

break exits the loop prematurely. **continue** skips the rest of the current iteration and moves to the next iteration.

Syntax:

```
1 for: # Code to repeat
2     if # boolean statement
3         break
4
5 for: # Code to repeat
6     if # boolean statement
7         continue
```

Example 1:

```
1 for num in range(1, 6):
2     if num == 3:
3         break
4     print(num)
```

Example 2:

```
1 for num in range(1, 6):
2     if num == 3:
3         continue
4     print(num)
```

NOT

Returns **True** if variable is **False**, and vice versa.

Syntax:

```
1 not variable
```

Example:

```
1 isLocked = False
2 print(not isLocked)
```

Not Equal(!=)

Checks if two values are not equal.

Syntax:

```
1 variable1 != variable2
```

Example 1:

```
1 a = 10
2 b = 20
3 a != b
```

returns **True**

Example 2:

```
1 count=0
2 count != 0
```

returns **False**

Object Creation

Creates an instance of a class (object) using the class constructor.

Syntax:

```
1 object_name = ClassName(arguments)
```

Example:

```
1 person1 = Person("Alice", 25)
```

OR

Returns **True** if either statement1 or statement2 (or both) are True. Otherwise, returns **False**.

Syntax:

```
1 statement1 or statement2
```

Example:

```
1 "Farewell Party Invitation"
2 grade = 12
3 if grade == 11 or grade == 12:
4     print("Farewell Party Invitation")
5 else:
6     print("Not eligible")
```

range()

Generates a sequence of numbers within a specified range.

Syntax:

```
1 range(stop)
2 range(start, stop)
3 range(start, stop, step)
```

Example:

```
1 range(5) #generates a sequence of integers from 0 to 4.
2 range(2, 10) #generates a sequence of integers from 2 to 9.
3 range(1, 11, 2) #generates odd integers from 1 to 9.
```

Return Statement

Return is a keyword used to send a value back from a function to its caller.

Syntax:

```
1 return value
```

Example:

```
1 def add(a, b): return a + b
2 result = add(3, 5)
```

Try-Except Block

Tries to execute the code in the try block. If an exception of the specified type occurs, the code in the except block is executed.

Syntax:

```
1 try: # Code that might raise an exception except
2 ExceptionType: # Code to handle the exception
```

Example:

```
1 try:
2     num = int(input("Enter a number: "))
3 except ValueError:
4     print("Invalid input. Please enter a valid number.")
```

Try-Except with Else Block

Code in the `else` block is executed if no exception occurs in the try block.

Syntax:

```
1 try: # Code that might raise an exception except
2     ExceptionType: # Code to handle the exception
3 else: # Code to execute if no exception occurs
```

Example:

```
1 try:
2     num = int(input("Enter a number: "))
3 except ValueError:
4     print("Invalid input. Please enter a valid number")
5 else:
6     print("You entered:", num)
```

Try-Except with Finally Block

Code in the **finally** block always executes, regardless of whether an exception occurred.

Syntax:

```
1 try: # Code that might raise an exception except
2     ExceptionType: # Code to handle the exception
3 finally: # Code that always executes
```

Example:

```
1 try:
2     file = open("data.txt", "r")
3     data = file.read()
4 except FileNotFoundError:
5     print("File not found.")
6 finally:
7     file.close()
```

While Loop

A **while** loop repeatedly executes a block of code as long as a specified condition remains **True**.

Syntax:

```
1 while condition: # Code to repeat
```

Example:

```
1 count = 0
2 while count < 5:
3     print(count)
4     count += 1
```

Module 3 Glossary: Python Programming Fundamentals

Term	Definition
Analogy	Refers to a concept or comparison outside the scope of the programming language itself, used to explain or relate one concept to another in a more understandable way.
Attributes	Attributes in Python refer to the characteristics or properties of an object, and they can be accessed using dot notation.
Branching	Branching in Python is a process of altering the flow of a program based on conditions, typically using if, elif, and else statements.

Comparison operators	Comparison operators in Python are used to compare values and return Boolean results (True or False), including operators like == (equal), != (not equal), < (less than), > (greater than), <= (less than or equal to), and >= (greater than or equal to).
Conditions	Conditions in Python are used to make decisions in code, executing specific blocks of code based on whether a given expression evaluates to True or False.
Enumerate	In Python, "enumerate" is a built-in function that adds a counter to an iterable, allowing you to loop through both the elements and their corresponding indices.
Exception handling	Exception handling in Python is a mechanism for gracefully managing and responding to errors or exceptional conditions that may occur during program execution.
Explicitly	In Python, the term "explicitly" refers to performing an action or specifying something in a clear, unambiguous, and direct manner.
For loops	For loops in Python are used for iterating over a sequence (such as a list, tuple, or string) or other iterable objects, executing a set of statements for each item in the sequence.
Global variable	Global variables in Python are variables defined outside of any function or block and can be accessed and modified from any part of the code.

Incremented	"Incremented" in Python means to increase the value of a variable by a specified amount, typically done using the += operator or by adding a fixed value.
Indent	In Python, "indent" refers to the use of whitespace at the beginning of a line to signify the structure and scope of code blocks, such as loops and functions.
Indices	In Python, "indices" refer to the position or location of elements in a sequence, like a string, list, or tuple, starting with 0 for the first element.
Iterate	In Python, "iterate" means to repeatedly perform a set of operations or steps on each item in a collection, such as a list, tuple, or dictionary, typically using loops or iterators.
Local variables	Local variables in Python are variables defined within a specific function or block of code and are only accessible within that function or block.
Logic operators	Logic operators in Python are used to perform logical operations on Boolean values, including operators like and (logical AND), or (logical OR), and not (logical NOT).
Loops	Loops in Python are constructs for repeating a block of code, enabling the execution of the same code multiple times.
Parameters	Parameters in Python are placeholders in a function definition, used to accept and work with values provided to the function when it is called.

Programming Fundamentals	Programming fundamentals in Python involve variables, control structures, functions, data structures, input/output, and error handling for building software.
Range function	The range function in Python generates a sequence of numbers that can be used for iterating in a loop and is typically used as range (start, stop, step), where it creates numbers from start to stop-1 with the given step increment.
Scope of function	The "scope of a function" in Python refers to the region of code where a variable defined within that function is accessible or visible.
Sequences	Sequences in Python are ordered collections of items that can include data types like strings, lists, and tuples, allowing for indexing and iteration.
Syntax	In Python, "Syntax" refers to the set of rules that dictate how code must be written and structured to be correctly interpreted by the Python interpreter. It includes correct use of keywords, indentation, operators, and punctuation.
While loops	While loops in Python are used to repeatedly execute a block of code as long as a specified condition is true.

Module 4: Working with Data in Python

Reading and Writing Files with Open

Reading Files with Open

Introduction

Reading text files involves extracting and processing the data stored within them. Text files can have various structures, and how you read them depends on their format. Here's a general guide on reading text files with different structures.

Plain text files

- Plain text files contain unformatted text without any specific structure
- You can read plain text files line by line or load all the content into your memory

Opening the file

There are two methods for opening the file using the file handling concept.

1. Using Python's open function

Suppose we have a file named **'file.txt'**.

Python's `open` function creates a file object and accesses the data within a text file. It takes two primary parameters:

1. **File path:** The file path parameter consists of the filename and directory where the file is located.
2. **Mode:** The mode parameter specifies the purpose of opening the file, such as 'r' for reading, 'w' for writing, or 'a' for appending.

```
1 # Open the file in read ('r') mode
2 file = open('file.txt', 'r')
```


`open('file.txt', 'r')`: This line opens a file named 'file.txt' in read mode ('r'). It returns a file object, which is stored in the variable file. The 'r' mode indicates that the file will be opened for reading.

2. Using 'with' statement

To simplify file handling and ensure proper closure of files, Python provides the "with" statement. It automatically closes the file when operations within the indented block are completed. This is considered best practice when working with files.

```
1 # Open the file using 'with' in read ('r') mode
2 with open('file.txt', 'r') as file:
3     # further code
```

Open the file using 'with' in read ('r') mode

`with open('file.txt', 'r') as file:` This line opens a file named 'file.txt' in read mode ('r') using the with statement, which is a context manager. The file is automatically closed when the code block inside the with statement exits.

Advantages of using the with statement

The key advantages of using the 'with' statement are:

- **Automatic resource management:** The file is guaranteed to be closed when you exit the with block, even if an exception occurs during processing.
- **Cleaner and more concise code:** You don't need to explicitly call `close()`, making your code more readable and less error-prone.

Note: For most file reading and writing operations in Python, the 'with' statement is recommended.

Let's perform a read operation on a file

1. Reading the entire content

You can read the entire content of a file using the read method, which stores the data as a string in a variable. This content can be printed or further manipulated as needed.

```
1  # Reading and Storing the Entire Content of a File
2
3  # Using the read method, you can retrieve the complete content of a file
4  # and store it as a string in a variable for further processing or display.
5
6  # Step 1: Open the file you want to read
7  with open('file.txt', 'r') as file:
8
9      # Step 2: Use the read method to read the entire content of the file
10     file_stuff = file.read()
11
12     # Step 3: Now that the file content is stored in the variable 'file_stuff',
13     # you can manipulate or display it as needed.
14
15     # For example, let's print the content to the console:
16     print(file_stuff)
17
18 # Step 4: The 'with' statement automatically closes the file when it's done,
19 # ensuring proper resource management and preventing resource leaks.
```

Step 1: Involves opening the file, specifying 'file.txt' as the file to be opened for reading ('r') mode using the with context manager.

Step 2: Utilizes the `read()` statement on the file object (file) to read the entire file. This content is then stored in the file_stuff variable.

Step 3: Explain that with the content now stored in file_stuff, you can perform various operations on it. In the example provided, the code prints the content to the console, but you can manipulate, analyze, search, or process the text data in file_stuff based on your specific needs.

Step 4: Emphasizes that the with block automatically closes the file when done, ensuring proper resource management and preventing resource leaks. This is a crucial aspect of using the with statement when working with files.

2. Reading the content line by line

Python provides methods to read files line by line:

- The 'readlines' method reads the file line by line and stores each line as an element in a list. The order of lines in the list corresponds to their order in the file.
- The 'readline' method reads individual lines from the file. It can be called multiple times to read subsequent lines.

In Python, the readline() method is like reading a book one line at a time. Imagine you have a big book and want to read it page by page. readline() helps you do just that with lines of text instead of pages.

Here's how it works:

Opening a file: First, you need to open the file you want to read using the open() function.

```
1 file = open('file.txt', 'r')
```

Reading line by line: Now, you can use readline() to read one line from the file at a time. It's like turning the pages of the book, but here, you're getting one sentence (or line) at each turn.

```
1 line1 = file.readline() # Reads the first line
2 line2 = file.readline() # Reads the second line
```

Using the lines: You can do things with each line you read. For example, you can print it, check if it contains specific words, or save it somewhere else.

```
1 print(line1) # Print the first line
2 if 'important' in line2:
3     print('This line is important!')
```

Looping through lines: Typically, you use a loop to read lines until no more lines are left. It's like reading the entire book, line by line.

```
1 while True:
2     line = file.readline()
3     if not line:
4         break # Stop when there are no more lines to read
5     print(line)
```

Closing the book: When you're done reading, it's essential to close the file using `file.close()` to make sure you're not wasting resources.

```
1 file.close()
```

So, In simple terms, **`readline()`** helps you read a text file line by line, allowing you to work with each line of text as you go. It's like taking one sentence at a time from a book and doing something with it before moving on to the next sentence. Don't forget to close the book when you're done!

3. Reading specific characters

You can specify the number of characters to read using the `readlines` method. For example, reading the first four characters, the next five, and so on.

Reading specific characters from a text file in Python involves opening the file, navigating to the desired position, and then reading the characters you need. Here's a detailed explanation of how to read specific characters from a file:

Open the File

First, you need to open the file you want to read. Use the `open()` function with the appropriate file path and mode. For reading, use 'r' mode.

```
1 file = open('file.txt', 'r')
```

Navigate to the intended position (Optional)

If you want to read characters from a specific position in the file, you can use the `seek()` method. This method moves the file pointer (like a cursor) to a particular

position. The position is specified in bytes, so you'll need to know the byte offset of the characters you want to read.

```
1 file.seek(10) # Move to the 11th byte (0-based index)
```

Read specific characters

To read specific characters, you can use the `read()` method with an argument that specifies the number of characters to read. It reads characters starting from the current position of the file pointer.

```
1 characters = file.read(5) # Read the next 5 characters
```

In this example, it reads the next 5 characters from the current position of the file pointer.

Use the read characters

You can now use the `characters` variable to work with the specific characters you've read. You can print them, save them, manipulate them, or perform any other actions.

```
1 print(characters)
```

Close the file

It's essential to close the file when you're done to free up system resources and ensure proper file handling.

```
1 file.close()
```

Conclusion

In conclusion, this reading has provided a comprehensive overview of file handling in Python, with a focus on reading text files. File handling is a fundamental aspect of programming, and Python offers powerful built-in functions and methods to interact with files seamlessly.

Writing Files with Open

Writing to a file

You can create a new text file and write data to it using Python's `open()` function.

The `open()` function takes two main arguments: the file path (including the file name) and the mode parameter, which specifies the operation you want to perform on the file. For writing, you should use the mode 'w' Here's an example:

```
1 # Create a new file Example2.txt for writing
2 with open('Example2.txt', 'w') as file1:
3     file1.write("This is line A\n")
4     file1.write("This is line B\n")
5     # file1 is automatically closed when the 'with' block exits
```

Line 2 explanation:

```
with open('Example2.txt', 'w') as file1
```

- We start by using the `open` function to open or create a file named `Example2.txt` for writing ('w' mode).
- The `'w'` mode specifies that we intend to write data to the file.
- We use the `with` statement to ensure that the file is automatically closed when the code block exits. This helps manage resources efficiently.

Line 3 explanation:

```
file1.write("This is line A\n")
```

- Here, we use the `write()` method on the file object, `file1`, to add the text `This is line A` to the file.
- The `\n` at the end represents a newline character, which starts a new line in the file.

Line 4 explanation

```
file1.write("This is line" B\n")
```

- Similarly, we use the `write()` method again to add the text `This is line B` to the file on a new line.

Writing multiple lines to a file using a list and loop

In Python, you can use a list to store multiple lines of text and then write these lines to a file using a loop. Here's an example code snippet that demonstrates this:

```
1 # List of lines to write to the file
2 Lines = ["This is line 1", "This is line 2", "This is line 3"]
3
4 # Create a new file Example3.txt for writing
5 with open('Example3.txt', 'w') as file2:
6     for line in Lines:
7         file2.write(line + "\n")
8     # file2 is automatically closed when the 'with' block exits
```

Here's an explanation of the code:

- **Line 2:** We start by defining a list called `Lines`, which contains multiple lines of text that we want to write to the file. Each line is a string.
- **Line 5:** Next, we use the `open()` function to create a new text file named `Example3.txt` for writing, `'w'` mode. The `'w'` mode indicates that we intend to write data to the file.
- **Line 6:** We then enter a for loop to iterate through each element (line) in the `Lines` list.
- **Line 7:** Inside the loop, we use the `write()` method on the file object `file2` to write the current line of text (line) to the file. We add `\n` at the end of each line to ensure that each line is followed by a newline character, which separates them in the file.
- **Line 8:** Finally, we add a comment indicating that the file `file2` will be automatically closed when the code block within the with statement exits. Properly closing the file is essential for good resource management.

Appending data to an existing file

In Python, you can use the `'a'` mode when opening a file to append new data to an existing file without overwriting its contents. Here's an example code snippet that demonstrates this:

```
1 # Data to append to the existing file
2 new_data = "This is line C"
3
4 # Open an existing file Example2.txt for appending
5 with open('Example2.txt', 'a') as file1:
6     file1.write(new_data + "\n")
7     # file1 is automatically closed when the 'with' block exits
```

Here's an explanation of the code:

- **Line 2:** We start by defining a variable `new_data` that contains the text we want to append to the existing file. In this case, it's the string `This is line C`.
- **Line 5:** Next, we use the `open()` function to open an existing file named `Example2.txt` for appending, `'a'` mode. The `'a'` mode indicates that we intend to append data to the file, and if the file doesn't exist, it will be created.
- **Line 6:** Within the `with` block, we use the `write()` method on the file object `file1` to append the `new_data` to the file. We add `"\n"` at the end to ensure that the appended data starts on a new line, maintaining the file's readability.
- Finally, we add a comment indicating that the file `file1` will automatically close when the code block within the `with` statement exits. Properly closing the file is essential for good resource management.

Copying contents from one file to another

In Python, you can copy the contents of one file to another by reading from the source file and writing to the destination file. Here's an example code snippet that demonstrates this:


```

1  # Open the source file for reading
2  with open('source.txt', 'r') as source_file:
3      # Open the destination file for writing
4      with open('destination.txt', 'w') as destination_file:
5          # Read lines from the source file and copy them to the destination file
6          for line in source_file:
7              destination_file.write(line)
8      # Destination file is automatically closed when the 'with' block exits
9  # Source file is automatically closed when the 'with' block exits

```

Here's an explanation of the code:

- **Line 2:** We start by opening the source file, `source.txt` for reading, `r` mode, using the `with` statement and the `open()` function. This allows us to read data from the source file.
- **Line 4:** Inside the first `with` block, we open the destination file, `destination.txt` for writing, `w` mode, using another `with` statement and the `open()` function. This prepares the destination file for writing.
- **Line 6:** We use a `for` loop to iterate through each line in the source file `source_file`. This loop reads each line from the source file one by one.
- **Line 7:** Within the loop, we use the `write()` method to write each line from the source file to the destination file `destination_file`. This effectively copies the content of the source file to the destination file.
- **Lines 8 and 9:** After copying all the lines, both the source and destination files are automatically closed when their respective `with` blocks exit. Proper file closure is crucial for managing resources efficiently.

File modes in Python (syntax and use cases)

The following table provides an overview of different file modes, their syntax, and common use cases. Understanding these modes is essential when working with files in Python for various data manipulation tasks.

Mode	Syntax	Description

'r'	'r'	Read mode. Opens an existing file for reading. Raises an error if the file doesn't exist.
'w'	'w'	Write mode. Creates a new file for writing. Overwrites the file if it already exists.
'a'	'a'	Append mode. Opens a file for appending data. Creates the file if it doesn't exist.
'x'	'x'	Exclusive creation mode. Creates a new file for writing but raises an error if the file already exists.
'rb'	'rb'	Read binary mode. Opens an existing binary file for reading.
'wb'	'wb'	Write binary mode. Creates a new binary file for writing.
'ab'	'ab'	Append binary mode. Opens a binary file for appending data.
'xb'	'xb'	Exclusive binary creation mode. Creates a new binary file for writing but raises an error if it already exists.
'rt'	'rt'	Read text mode. Opens an existing text file for reading. (Default for text files)
'wt'	'wt'	Write text mode. Creates a new text file for writing. (Default for text files)
'at'	'at'	Append text mode. Opens a text file for appending data. (Default for text files)

'xt'	'xt'	Exclusive text creation mode. Creates a new text file for writing but raises an error if it already exists.
'r+'	'r+'	Read and write mode. Opens an existing file for both reading and writing.
'w+'	'w+'	Write and read mode. Creates a new file for reading and writing. Overwrites the file if it already exists.
'a+'	'a+'	Append and read mode. Opens a file for both appending and reading. Creates the file if it doesn't exist.
'x+'	'x+'	Exclusive creation and read/write mode. Creates a new file for reading and writing but raises an error if it already exists.

Conclusion

Working with files is a fundamental aspect of programming, and Python provides powerful tools to perform various file operations. In this summary, we covered key concepts and code examples related to file handling in Python, including writing, appending, and copying files.

PANDAS

What is Pandas?

Pandas is a popular open-source data manipulation and analysis library for the Python programming language. It provides a powerful and flexible set of tools for working with structured data, making it a fundamental tool for data scientists, analysts, and engineers. Pandas is designed to handle data in various formats, such as tabular data, time series data, and more, making it an essential part of the data processing workflow in many industries.

Key features and functionalities of Pandas

Data Structures: Pandas offers two primary data structures - DataFrame and Series.

1. A DataFrame is a two-dimensional, size-mutable, and potentially heterogeneous tabular data structure with labeled axes (rows and columns).
2. A Series is a one-dimensional labeled array, essentially a single column or row of data.

Data Import and Export: Pandas makes it easy to read data from various sources, including CSV files, Excel spreadsheets, SQL databases, and more. It can also export data to these formats, enabling seamless data exchange.

Data Merging and Joining: You can combine multiple DataFrames using methods like merge and join, similar to SQL operations, to create more complex datasets from different sources.

Efficient Indexing: Pandas provides efficient indexing and selection methods, allowing you to access specific rows and columns of data quickly.

Custom Data Structures: You can create custom data structures and manipulate data in ways that suit your specific needs, extending Pandas' capabilities.

Importing Pandas

Import Pandas using the import command, followed by the library's name. Commonly, Pandas is imported as pd for brevity in code.

```
1 import pandas as pd
```

Data Loading

- Pandas can be used to load data from various sources, such as CSV and Excel files.

- The `read_csv` function is used to load data from a CSV file into a Pandas DataFrame.

To read a CSV (Comma-Separated Values) file in Python using the Pandas library, you can use the `pd.read_csv()` function. Here's the syntax to read a CSV file:

```
1 import pandas as pd
2
3 # Read the CSV file into a DataFrame
4 df = pd.read_csv('your_file.csv')
```

Replace `'your_file.csv'` with the actual file path of your CSV file. Make sure that the file is located in the same directory as your Python script, or you provide the correct file path.

What is a Series?

A Series is a one-dimensional labeled array in Pandas. It can be thought of as a single column of data with labels or indices for each element. You can create a Series from various data sources, such as lists, NumPy arrays, or dictionaries. Here's a basic example of creating a Series in Pandas:

```
1 import pandas as pd
2 # Create a Series from a list
3 data = [10, 20, 30, 40, 50]
4 s = pd.Series(data)
5 print(s)
```

In this example, we've created a Series named `s` with numeric data. Notice that Pandas automatically assigned numerical indices (0, 1, 2, 3, 4) to each element, but you can also specify custom labels if needed.

Accessing Elements in a Series

You can access elements in a Series using the index labels or integer positions. Here are a few common methods for accessing Series data:

Accessing by label

```
1 print(s[2]) # Access the element with label 2 (value 30)
```

Accessing by position

```
1 print(s.iloc[3]) # Access the element at position 3 (value 40)
```

Accessing multiple elements

```
1 print(s[1:4]) # Access a range of elements by label
```

Series Attributes and Methods

Pandas Series come with various attributes and methods to help you manipulate and analyze data effectively. Here are a few essential ones:

- **values**: Returns the Series data as a NumPy array.
- **index**: Returns the index (labels) of the Series.
- **shape**: Returns a tuple representing the dimensions of the Series.
- **size**: Returns the number of elements in the Series.
- **mean()**, **sum()**, **min()**, **max()**: Calculate summary statistics of the data.
- **unique()**, **nunique()**: Get unique values or the number of unique values.
- **sort_values()**, **sort_index()**: Sort the Series by values or index labels.
- **isnull()**, **notnull()**: Check for missing (NaN) or non-missing values.
- **apply()**: Apply a custom function to each element of the Series.

What is a DataFrames?

A DataFrame is a two-dimensional labeled data structure with columns of potentially different data types. Think of it as a table where each column represents a variable, and each row represents an observation or data point. DataFrames are suitable for a wide range of data, including structured data from CSV files, Excel spreadsheets, SQL databases, and more.

Creating DataFrames from Dictionaries

DataFrames can be created from dictionaries, with keys as column labels and values as lists representing rows.

```
1 import pandas as pd
2
3 # Creating a DataFrame from a dictionary
4 data = {'Name': ['Alice', 'Bob', 'Charlie', 'David'],
5         'Age': [25, 30, 35, 28],
6         'City': ['New York', 'San Francisco', 'Los Angeles', 'Chicago']}
7
8 df = pd.DataFrame(data)
9
10 print(df)
11
```

Column Selection

You can select a single column from a DataFrame by specifying the column name within double brackets. Multiple columns can be selected in a similar manner, creating a new DataFrame.

```
1 print(df[['Name']]) # Access the 'Name' column
```

Accessing Rows

You can access rows by their index using `.iloc[]` or by label using `.loc[]`.

```
1 print(df.iloc[2]) # Access the third row by position
2 print(df.loc[1])  # Access the second row by label
```

Slicing

You can slice DataFrames to select specific rows and columns.

```
1 print(df[['Name', 'Age']]) # Select specific columns
2 print(df[1:3])             # Select specific rows
```

Finding Unique Elements

Use the unique method to determine the unique elements in a column of a DataFrame.

```
1 unique_dates = df['Age'].unique()
```

Conditional Filtering

You can filter data in a DataFrame based on conditions using inequality operators. For instance, you can filter albums released after a certain year.

```
1 high_above_102 = df[df['Age'] > 25]
```

Saving DataFrames

To save a DataFrame to a CSV file, use the to_csv method and specify the filename with a “.csv” extension. Pandas provides other functions for saving DataFrames in different formats.

```
1 df.to_csv('trading_data.csv', index=False)
```

DataFrame Attributes and Methods

DataFrames provide numerous attributes and methods for data manipulation and analysis, including:

- **shape**: Returns the dimensions (number of rows and columns) of the DataFrame.
- **info()**: Provides a summary of the DataFrame, including data types and non-null counts.
- **describe()**: Generates summary statistics for numerical columns.
- **head()**, **tail()**: Displays the first or last n rows of the DataFrame.
- **mean()**, **sum()**, **min()**, **max()**: Calculate summary statistics for columns.
- **sort_values()**: Sort the DataFrame by one or more columns.

- **groupby()**: Group data based on specific columns for aggregation.
- **fillna(), drop(), rename()**: Handle missing values, drop columns, or rename columns.
- **apply()**: Apply a function to each element, row, or column of the DataFrame.

Pandas offers a wide range of methods beyond these examples. For more detailed information, please refer to the official documentation available on the [Pandas official website](#).

Conclusion

In conclusion, mastering the use of Pandas Series and DataFrames is essential for effective data manipulation and analysis in Python. Series provide a foundation for handling one-dimensional data with labels, while DataFrames offer a versatile, table-like structure for working with two-dimensional data. Whether you're cleaning, exploring, transforming, or analyzing data, these Pandas data structures, along with their attributes and methods, empower you to efficiently and flexibly manipulate data to derive valuable insights. By incorporating Series and DataFrames into your data science toolkit, you'll be well-prepared to tackle a wide range of data-related tasks and enhance your data analysis capabilities. To further your skills in data analysis with Pandas, consider the following next steps:

- Work with real datasets to apply what you've learned and gain hands-on experience.
- Visit the [Pandas official website](#) to explore the extensive documentation and discover more functions and methods.

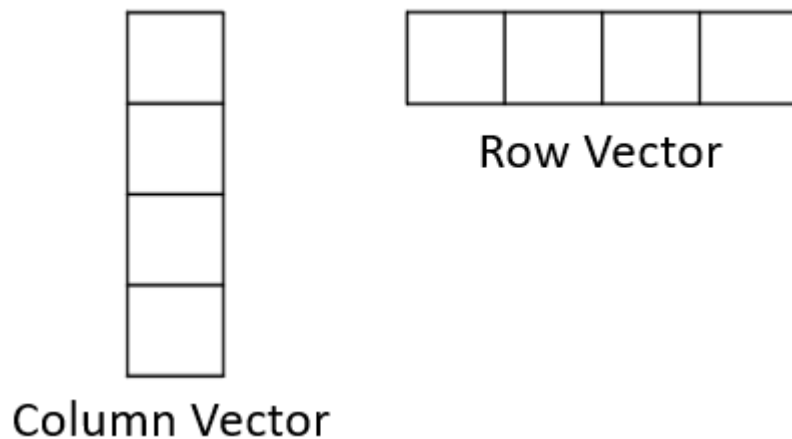
Numpy in Python

Matrix Mathematics

You have seen that you can use Numpy package functions to perform different types of operations on arrays and matrices. In this reading, you will learn how these operations work mathematically.

1D Arrays : Vectors

A 1D array is often termed as a vector. Depending upon the orientation of the data, the vector can be classified as a row vector or a column vector. This is illustrated in the image below.



Mathematically, we can add, subtract, and take the product of two vectors, provided they are the same shape. The images below highlight the mathematical operations conducted on a pair of vectors.

$$\begin{array}{|c|} \hline A1 \\ \hline A2 \\ \hline A3 \\ \hline A4 \\ \hline \end{array}
 +
 \begin{array}{|c|} \hline B1 \\ \hline B2 \\ \hline B3 \\ \hline B4 \\ \hline \end{array}
 =
 \begin{array}{|c|} \hline A1+B1 \\ \hline A2+B2 \\ \hline A3+B3 \\ \hline A4+B4 \\ \hline \end{array}$$

Vector addition

$$\begin{array}{|c|} \hline A1 \\ \hline A2 \\ \hline A3 \\ \hline A4 \\ \hline \end{array}
 -
 \begin{array}{|c|} \hline B1 \\ \hline B2 \\ \hline B3 \\ \hline B4 \\ \hline \end{array}
 =
 \begin{array}{|c|} \hline A1-B1 \\ \hline A2-B2 \\ \hline A3-B3 \\ \hline A4-B4 \\ \hline \end{array}$$

Vector Subtraction

$$\begin{array}{|c|} \hline A1 \\ \hline A2 \\ \hline A3 \\ \hline A4 \\ \hline \end{array}
 \cdot
 \begin{array}{|c|} \hline B1 \\ \hline B2 \\ \hline B3 \\ \hline B4 \\ \hline \end{array}
 =
 \begin{array}{|c|} \hline A1 \cdot B1 \\ \hline A2 \cdot B2 \\ \hline A3 \cdot B3 \\ \hline A4 \cdot B4 \\ \hline \end{array}$$

Vector Product

All three of these operations are conducted on corresponding elements of individual vectors. The resulting array always has the same size as that of the two original vectors.

To a single vector, we can also add a constant (scalar addition), subtract a constant (scalar subtraction) and multiply a constant (scalar multiplication) to any vector. The images below illustrate these operations.

$$\begin{array}{|c|} \hline A1 \\ \hline A2 \\ \hline A3 \\ \hline A4 \\ \hline \end{array} + C = \begin{array}{|c|} \hline A1+C \\ \hline A2+C \\ \hline A3+C \\ \hline A4+C \\ \hline \end{array}$$

Scalar addition

$$\begin{array}{|c|} \hline A1 \\ \hline A2 \\ \hline A3 \\ \hline A4 \\ \hline \end{array} - C = \begin{array}{|c|} \hline A1-C \\ \hline A2-C \\ \hline A3-C \\ \hline A4-C \\ \hline \end{array}$$

Scalar Subtraction

$$\begin{array}{|c|} \hline A1 \\ \hline A2 \\ \hline A3 \\ \hline A4 \\ \hline \end{array} \times C = \begin{array}{|c|} \hline A1 \times C \\ \hline A2 \times C \\ \hline A3 \times C \\ \hline A4 \times C \\ \hline \end{array}$$

Scalar Product

2D Arrays : Matrices

A 2D array is also called a Matrix. These are typically rectangular arrays with data stored in different rows. All of the operations mentioned above are also applicable to the 2D arrays.

As illustrated in the images below, the dot product is carried out by multiplying and adding corresponding elements of rows of the first matrix with the elements of

columns of the second matrix. As a result, the output matrix from the multiplication will have a modified shape.

The general rule is that the dot product of an $m \times n$ matrix can be done only with an $n \times p$ matrix, and the resultant matrix will have the shape $m \times p$. In the example shown below, the 4×2 matrix is multiplied with the 2×4 matrix to generate a 4×4 matrix.

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \\ A_{31} & A_{32} \\ A_{41} & A_{42} \end{bmatrix} \times \begin{bmatrix} B_{11} & B_{12} & B_{13} & B_{14} \\ B_{21} & B_{22} & B_{23} & B_{24} \end{bmatrix} = \begin{bmatrix} A_{11}B_{11} + A_{12}B_{21} & A_{11}B_{12} + A_{12}B_{22} & A_{11}B_{13} + A_{12}B_{23} & A_{11}B_{14} + A_{12}B_{24} \\ A_{21}B_{11} + A_{22}B_{21} & A_{21}B_{12} + A_{22}B_{22} & A_{21}B_{13} + A_{22}B_{23} & A_{21}B_{14} + A_{22}B_{24} \\ A_{31}B_{11} + A_{32}B_{21} & A_{31}B_{12} + A_{32}B_{22} & A_{31}B_{13} + A_{32}B_{23} & A_{31}B_{14} + A_{32}B_{24} \\ A_{41}B_{11} + A_{42}B_{21} & A_{41}B_{12} + A_{42}B_{22} & A_{41}B_{13} + A_{42}B_{23} & A_{41}B_{14} + A_{42}B_{24} \end{bmatrix}$$

In the reverse example, when 2×4 matrix is multiplied with the 4×2 one, the resultant will be a 2×2 matrix.

$$\begin{bmatrix} B_{11} & B_{12} & B_{13} & B_{14} \\ B_{21} & B_{22} & B_{23} & B_{24} \end{bmatrix} \times \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \\ A_{31} & A_{32} \\ A_{41} & A_{42} \end{bmatrix} = \begin{bmatrix} B_{11}A_{11} + B_{12}A_{21} + B_{13}A_{31} + B_{14}A_{41} & B_{11}A_{12} + B_{12}A_{22} + B_{13}A_{32} + B_{14}A_{42} \\ B_{21}A_{11} + B_{22}A_{21} + B_{23}A_{31} + B_{24}A_{41} & B_{21}A_{12} + B_{22}A_{22} + B_{23}A_{32} + B_{24}A_{42} \end{bmatrix}$$

Note: Dot product of a row vector with a column vector, with the same number of elements, would return a single scalar value. Dot product of a column vector with a row vector, will return a 2D matrix.

Beginner's Guide to NumPy

What is NumPy?

NumPy, short for **N**umerical **P**ython, is a fundamental library for numerical and scientific computing in Python. It provides support for large, multi-dimensional arrays and matrices, along with a collection of high-level mathematical functions to operate on these arrays. NumPy serves as the foundation for many data science and machine learning libraries, making it an essential tool for data analysis and scientific research in Python.

Key aspects of NumPy in Python:

- **Efficient data structures:** NumPy introduces efficient array structures, which are faster and more memory-efficient than Python lists. This is crucial for handling large data sets.
- **Multi-dimensional arrays:** NumPy allows you to work with multi-dimensional arrays, enabling the representation of matrices and tensors. This is particularly useful in scientific computing.
- **Element-wise operations:** NumPy simplifies element-wise mathematical operations on arrays, making it easy to perform calculations on entire data sets in one go.
- **Random number generation:** It provides a wide range of functions for generating random numbers and random data, which is useful for simulations and statistical analysis.
- **Integration with other libraries:** NumPy seamlessly integrates with other data science libraries like SciPy, Pandas, and Matplotlib, enhancing its utility in various domains.
- **Performance optimization:** NumPy functions are implemented in low-level languages like C and Fortran, which significantly boosts their performance. It's a go-to choice when speed is essential.

Installation

If you haven't already installed NumPy, you can do so using `pip`:

```
1 pip install numpy
```

Creating NumPy arrays

You can create NumPy arrays from Python lists. These arrays can be one-dimensional or multi-dimensional.

Creating 1D array

```
1 import numpy as np
```

import numpy as np: In this line, the NumPy library is imported and assigned an alias `np` to make it easier to reference in the code.

```
1 # Creating a 1D array
2 arr_1d = np.array([1, 2, 3, 4, 5]) # **np.array()** is used to create NumPy arrays.
```

arr_1d = np.array([1, 2, 3, 4, 5]): In this line, a one-dimensional NumPy array named `arr_1d` is created. It uses the `np.array()` function to convert a Python list `[1, 2, 3, 4, 5]` into a NumPy array. This array contains five elements, which are 1, 2, 3, 4, and 5. `arr_1d` is a 1D array because it has a single row of elements.

Creating 2D array

```
1 import numpy as np
```

import numpy as np: In this line, the NumPy library is imported and assigned an alias `np` to make it easier to reference in the code.

```
1 # Creating a 2D array
2 arr_2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
```

arr_2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]]): In this line, a two-dimensional NumPy array named `arr_2d` is created. It uses the `np.array()` function to convert a list of lists into a 2D NumPy array. The outer list contains three inner lists, each of which represents a row of elements. So, `arr_2d` is a 2D array with three rows and three columns. The elements in this array form a matrix with values from 1 to 9, organized in a 3x3 grid.

Array attributes

NumPy arrays have several useful attributes:

```
1 # Array attributes
2 print(arr_2d.ndim) # ndim : Represents the number of dimensions or "rank" of the array.
3 # output : 2
4 print(arr_2d.shape) # shape : Returns a tuple indicating the number of rows and columns in
5 # Output : (3, 3)
6 print(arr_2d.size) # size: Provides the total number of elements in the array.
7 # Output : 9
```

Indexing and slicing

You can access elements of a NumPy array using indexing and slicing:

In this line, the third element (index 2) of the 1D array `arr_1d` is accessed.

```
1 # Indexing and slicing
2 print(arr_1d[2]) # Accessing an element (3rd element)
```

In this line, the element in the 2nd row (index 1) and 3rd column (index 2) of the 2D array `arr_2d` is accessed.

```
1 print(arr_2d[1, 2]) # Accessing an element (2nd row, 3rd column)
```

In this line, the 2nd row (index 1) of the 2D array `arr_2d` is accessed.

```
1 print(arr_2d[1, 2]) # Accessing an element (2nd row, 3rd column)
```

In this line, the 2nd column (index 1) of the 2D array `arr_2d` is accessed.

```
1 print(arr_2d[:, 1]) # Accessing a column (2nd column)
```

Basic operations

NumPy simplifies basic operations on arrays:

Element-wise arithmetic operations:

Addition, subtraction, multiplication, and division of arrays with scalars or other arrays.

Array addition

```
1 # Array addition
2 array1 = np.array([1, 2, 3])
3 array2 = np.array([4, 5, 6])
4 result = array1 + array2
5 print(result) # [5 7 9]
```

Scalar multiplication

```
1 # Scalar multiplication
2 array = np.array([1, 2, 3])
3 result = array * 2 # each element of an array is multiplied by 2
4 print(result) # [2 4 6]
```

Element-wise multiplication (Hadamard Product)

```
1 # Element-wise multiplication (Hadamard product)
2 array1 = np.array([1, 2, 3])
3 array2 = np.array([4, 5, 6])
4 result = array1 * array2
5 print(result) # [4 10 18]
```

Matrix multiplication

```
1 # Matrix multiplication
2 matrix1 = np.array([[1, 2], [3, 4]])
3 matrix2 = np.array([[5, 6], [7, 8]])
4 result = np.dot(matrix1, matrix2)
5 print(result)
6 # [[19 22]
7 #  [43 50]]
```

NumPy simplifies these operations, making it easier and more efficient than traditional Python lists.

Operation with NumPy

Here's the list of operation which can be performed using Numpy

Operation	Description	Example
Array Creation	Creating a NumPy array.	<code>arr = np.array([1, 2, 3, 4, 5])</code>
Element-Wise Arithmetic	Element-wise addition, subtraction, and so on.	<code>result = arr1 + arr2</code>
Scalar Arithmetic	Scalar addition, subtraction, and so on.	<code>result = arr * 2</code>
Element-Wise Functions	Applying functions to each element.	<code>result = np.sqrt(arr)</code>
Sum and Mean	Calculating the sum and mean of an array. Calculating the sum and mean of an array.	<code>total = np.sum(arr)</code> <code>average = np.mean(arr)</code>
Maximum and Minimum Values	Finding the maximum and minimum values.	<code>max_val = np.max(arr)</code> <code>min_val = np.min(arr)</code>
Reshaping	Changing the shape of an array.	<code>reshaped_arr = arr.reshape(2, 3)</code>

Transposition	Transposing a multi-dimensional array.	<pre>transposed_arr = arr.T</pre>
Matrix Multiplication	Performing matrix multiplication.	<pre>result = np.dot(matrix1, matrix2)</pre>

Conclusion

NumPy is a fundamental library for data science and numerical computations. This guide covers the basics of NumPy, and there's much more to explore. Visit numpy.org for more information and examples.

Module 4 Summary: Working with Data in Python

- Python uses the `open()` function and allows you to read and write files, providing access to the content within the file for reading. It also allows overwriting it for writing and specifies the file mode (for example, `r` for reading, `w` for writing, `a` for appending).
 - To read a file, Python uses an `open` function along with `r`.
 - Python uses the **open with** function to read and process a file attribute, that is, from open to close.
 - In Python, you use the **open** method to edit or overwrite a file.
 - To write a file, Python uses the **open** function along with `w`.
 - In Python, "`a`" indicates that the program has appended to the file.
 - In Python, "`\n`" signifies that the code should start on a new line.
 - Python uses various methods to print lines from attributes.

- Pandas is a powerful Python library for data manipulation and analysis, providing data structures and functions to work with structured data like data frames and series.
 - You import the file (panda) by using the import command followed by the file name.
 - In Python, you use the **as** command to provide a shorter name for the file.
 - In Pandas, you use a data frame (df) to specify the files to read.
 - DataFrames consist of rows and columns.
 - You can create new DataFrames by using the column or columns of a specific DataFrame.
 - We can work with data in a DataFrames and save the results in different formats.
 - In Python, you use the **Unique** method to determine unique elements in a column of the DataFrames.
 - You use the inequality operator along with df to assign a Boolean value to the selected column in DataFrames.
 - You save a new DataFrame as a different DataFrame, which may contain values from an earlier DataFrame.
- NumPy is a Python library for numerical and matrix operations, offering multidimensional array objects and a variety of mathematical functions to work with data efficiently.
 - NumPy is a basis for Pandas.
 - A NumPy array or ND array is similar to a list, usually of a fixed size with the same kind of element.
- A one-dimensional NumPy array is a linear sequence of elements with a single axis, like a traditional list, but optimized for numerical computations and array operations.
 - You can access elements in a NumPy using an index.
 - You use the attribute **dtype** to get the data type of the array elements.
 - You use **size** and **ndim** to get the size and dimension of the array, respectively.
 - You can use indexing and slicing methods in NumPy.
 - Vector additions are widely used operations in Python.

- Representing vector addition with line segments or arrows is useful.
- NumPy codes work much faster, which is helpful with lots of data.
- You perform vector subtraction by replacing the addition sign with a negative sign.
- Multiplying an array by a scalar in Python entails multiplying each element of the array by the scalar value, leading to a new array in which each element scales by the scalar.
- Hadamard product refers to the element-wise multiplication of two arrays of the same shape, resulting in a new array where each element is the product of the corresponding elements in the input arrays.
- The dot product in Python is the sum of the element-wise products of two arrays, often used for vector and matrix operations to find the scalar result of multiplying corresponding elements and summing them.
- When working with NumPy, it is common to utilize libraries like Matplotlib to create graphs and visualizations from numerical data stored in NumPy arrays.
- A two-dimensional NumPy array is a grid-like structure with rows and columns suitable for representing data as a matrix or a table for numerical computations.
 - In NumPy, "shape" refers to an array's dimensions (number of rows and columns), indicating its size and structure.
 - You use the attribute "size" to obtain the size of an array.
 - You use rectangular attributes to access the various elements in an array.
 - You use a scalar to multiply elements in NumPy.

Cheat Sheet: Working with Data in Python

Reading and writing files

File opening modes

Different modes to open files for specific operations.

Syntax: r (reading) w (writing) a (appending) + (updating: read/write) b (binary, otherwise text)

```
1 Examples: with open("data.txt", "r") as file: content = file.read()
print(content) with open("output.txt", "w") as file: file.write("Hello,
world!") with open("log.txt", "a") as file: file.write("Log entry:
Something happened.") with open("data.txt", "r+") as file: content =
file.read() file.write("Updated content: " + content)</td>
```

File reading methods

Different methods to read file content in various ways.

Syntax:

```
1 file.readlines() # reads all lines as a list
2 readline() # reads the next line as a string
3 file.read() # reads the entire file content as a string
```

Example:

```
1 with open("data.txt", "r") as file:
2     lines = file.readlines()
3     next_line = file.readline()
4     content = file.read()
```

File writing methods

Different write methods to write content to a file.

Syntax:

```
1 file.write(content) # writes a string to the file
2 file.writelines(lines) # writes a list of strings to the file
```

Example:

```
1 lines = ["Hello\n", "World\n"]
2 with open("output.txt", "w") as file:
3     file.writelines(lines)
```

Iterating over lines

Iterates through each line in the file using a `loop`.

Syntax:

```
1 for line in file: # Code to process each line
```

Example:

```
1 with open("data.txt", "r") as file:
2     for line in file: print(line)
```

open() and close()

Opens a file, performs operations, and explicitly closes the file using the `close()` method.

Syntax:

```
1 file = open(filename, mode) # Code that uses the file
2 file.close()
```

Example:

```
1 file = open("data.txt", "r")
2 content = file.read()
3 file.close()
```

with open()

Opens a file using a with block, ensuring automatic file closure after usage.

Syntax:

```
1 with open(filename, mode) as file: # Code that uses the file
```

Example:

```
1 with open("data.txt", "r") as file:
2     content = file.read()
```

Pandas

.read_csv()

Reads data from a .csv file and creates a DataFrame.

Syntax:

```
1 dataframe_name = pd.read_csv("filename.csv") Example: df = pd.read_csv("data.csv")
```

.read_excel()

Reads data from an Excel file and creates a DataFrame.

Syntax:

```
1 dataframe_name = pd.read_excel("filename.xlsx")
```

Example:

```
1 df = pd.read_excel("data.xlsx")
```

.to_csv()

Writes DataFrame to a CSV file.

Syntax:

```
1 dataframe_name.to_csv("output.csv", index=False)
```

Example:

```
1 df.to_csv("output.csv", index=False)
```

Access Columns

Accesses a specific column using `[]` in the DataFrame.

Syntax:

```
1 dataframe_name["column_name"] # Accesses single column
2 dataframe_name[["column1", "column2"]] # Accesses multiple columns
```

Example:

```
1 df["age"]
2 df[["name", "age"]]
```

describe()

Generates a statistics summary of numeric columns in the DataFrame.

Syntax:

```
1 dataframe_name.describe()
```

Example:

```
1 df.describe()
```

drop()

Removes specified rows or columns from the DataFrame. axis=1 indicates columns.
axis=0 indicates rows.

Syntax:

```
1 dataframe_name.drop(["column1", "column2"], axis=1, inplace=True)
2 dataframe_name.drop(index=[row1, row2], axis=0, inplace=True)
```

Example:

```
1 df.drop(["age", "salary"], axis=1, inplace=True) # Will drop columns
2 df.drop(index=[5, 10], axis=0, inplace=True) # Will drop rows
```

dropna()

Removes rows with missing NaN values from the DataFrame. axis=0 indicates rows.

Syntax:

```
1 dataframe_name.dropna(axis=0, inplace=True)
```

Example:

```
1 df.dropna(axis=0, inplace=True)
```

uplicated()

Identifies duplicate or repetitive values or records within a dataset.

Syntax:

```
1 dataframe_name.duplicated()
```

Example:

```
1 duplicate_rows = df[df.duplicated()]
```

Filter Rows

Creates a new DataFrame with rows that meet specified conditions.

Syntax:

```
1 filtered_df = dataframe_name[(Conditional_statements)]
```

Example:

```
1 filtered_df = df[(df["age"] > 30) & (df["salary"] < 50000)]
```

groupby()

Splits a DataFrame into groups based on specified criteria, enabling subsequent aggregation, transformation, or analysis within each group.

Syntax:

```
1 grouped = dataframe_name.groupby(by, axis=0, level=None, as_index=True,  
2 sort=True, group_keys=True, squeeze=False, observed=False, dropna=True)
```

Example:

```
1 grouped = df.groupby(["category", "region"]).agg({"sales": "sum"})
```

head()

Displays the first n rows of the DataFrame.

Syntax:

```
1 dataframe_name.head(n)
```

Example:

```
1 df.head(5)
```

Import pandas

Imports the Pandas library with the alias pd

Syntax:

```
1 import pandas as pd
```

Example:

```
1 import pandas as pd
```

info()

Provides information about the DataFrame, including data types and memory usage.

Syntax:

```
1 dataframe_name.info()
```

Example:

```
1 df.info()
```

merge()

Merges two DataFrames based on multiple common columns.

Syntax:

```
1 merged_df = pd.merge(df1, df2, on=["column1", "column2"])
```

Example:

```
1 merged_df = pd.merge(sales, products, on=["product_id", "category_id"])
```

print DataFrame

Displays the content of the DataFrame.

Syntax:

```
1 print(df) # or just type df
```

Example:

```
1 print(df)
2 df
```

replace()

Replaces specific values in a column with new values.

Syntax:

```
1 dataframe_name["column_name"].replace(old_value, new_value, inplace=True)
```

Example:

```
1 df["status"].replace("In Progress", "Active", inplace=True)
```

tail()

Displays the last n rows of the DataFrame.

Syntax:

```
1 dataframe_name.tail(n)
```

Example:

```
1 df.tail(5)
```

NumPy

Importing NumPy

Imports the NumPy library.

Syntax:

```
1 import numpy as np
```

Example:

```
1 import numpy as np
```

np.array()

Creates a one-dimensional or multi-dimensional array.

Syntax:

```
1 array_1d = np.array([list1 values]) # 1D Array
2 array_2d = np.array([[list1 values], [list2 values]]) # 2D Array
```

Example:

```
1 array_1d = np.array([1, 2, 3]) # 1D Array
2 array_2d = np.array([[1, 2], [3, 4]]) # 2D Array
```

NumPy Array Attributes

Common NumPy operations on arrays:

- Calculates the mean of array elements
- Calculates the sum of array elements
- Finds the minimum value in the array
- Finds the maximum value in the array
- Computes the dot product of two arrays

Example:

```
1 np.mean(array)
2 np.sum(array)
3 np.min(array)
4 np.max(array)
5 np.dot(array_1, array_2)
```

Module 4 Glossary: Working with Data in Python

Term	Definition
.csv file	A .csv (Comma-Separated Values) file is a plain text file format for storing tabular data, where each line represents a row and uses commas to separate values in different columns.
.txt file	A .txt (Text) file is a common file format that contains plain text without specific formatting, making it suitable for storing and editing textual data.
Append	To "append" means to add or attach something to the end of an existing object, typically used in the context of adding data to a file or elements to a data structure like a list in Python.
Attribute	An "attribute" in Python refers to a property or characteristic associated with an object, which can be accessed using dot notation.

Broadcasting in NumPy	Broadcasting in NumPy allows arrays with different shapes to be combined in element-wise operations by automatically extending smaller arrays to match the shape of larger ones, making operations more flexible.
Component	In NumPy, a "component" typically refers to a specific element or value within a multi-dimensional array, which can be accessed using indexing.
Computation	Computation in NumPy involves performing numerical operations on arrays and matrices, making it a powerful library for mathematical and scientific computing in Python.
Data analysis	Data analysis is the process of inspecting, cleaning, transforming, and interpreting data to discover useful information, draw conclusions, and support decision-making.
DataFrames	A DataFrames in Pandas is a two-dimensional, tabular data structure for storing and analyzing data, consisting of rows and columns.
Dependencies	Dependencies in Pandas are external libraries or modules, such as NumPy, that Pandas rely on for fundamental data manipulation and analysis functionality.
File attribute	File attributes generally refer to properties or metadata associated with files, which are managed at the operating system level.
File object	A "file object" in Python represents an open file, allowing reading from or writing to the file.

Grid	In Python, a "grid" typically refers to a two-dimensional structure composed of rows and columns, often used to represent data in a tabular format or for organizing objects in a coordinate system.
Hadamard Product	The Hadamard product is a mathematical operation that involves element-wise multiplication of two matrices or arrays of the same shape, producing a new matrix with each element being the product of the corresponding elements in the input matrices.
Importing pandas	To import Pandas in Python, you use the statement: <code>import pandas as pd</code> , which allows you to access Pandas functions and data structures using the abbreviation "pd."
Index	In Python, an "index" typically refers to a position or identifier used to access elements within a sequence or data structure, such as a list or string.
Libraries	Libraries in Python are collections of pre-written code modules that provide reusable functions and classes to simplify and enhance software development.
Linspace	In Python, "linspace" refers to a NumPy function that generates an array of evenly spaced values within a specified range.
NumPy	NumPy in Python is a fundamental library for numerical computing that provides support for large, multi-dimensional arrays and matrices, as well as a variety of high-level mathematical functions to operate on these arrays.

One dimensional NumPy	A one-dimensional NumPy array is a linear data structure that stores elements in a single sequence, often used for numerical computations and data manipulation.
Open function	In Python, the "open" function is used to access and manipulate files, allowing you to read from or write to a specified file.
Pandas	Pandas is a popular Python library for data manipulation and analysis, offering data structures and tools for working with structured data like tables and time series.
Pandas library	Pandas library in Python refer to the various modules and functions within the Pandas library, which provides powerful data structures and data analysis tools for working with structured data.
Plotting Mathematical Functions	Plotting mathematical functions in Python involves using libraries like Matplotlib to create graphical representations of mathematical equations, aiding visualization, and analysis.
Shape	In NumPy, "shape" refers to an array's dimensions (number of rows and columns), describing its size and structure.
Slicing	Slicing in NumPy entails extracting specific portions of an array by specifying a range of indices, enabling you to work with subsets of the data.
Two dimensional NumPy	A two-dimensional NumPy array is a structured data representation with rows and columns, resembling a matrix or table, ideal for various data manipulation and analysis tasks.

Universal Functions	Universal functions (ufuncs) in NumPy are functions that operate element-wise on arrays, providing efficient and vectorized operations for a wide range of mathematical and logical operations.
Vector addition	Vector addition in Python involves adding corresponding elements of two or more vectors, producing a new vector with the sum of their components.
Visualizations	Visualizations in Python refer to the creation of graphical representations, such as charts, plots, and graphs, to illustrate and communicate data and trends visually.

Module 5: APIs and Data Collection

Simple APIs

Some Context on APIs

What are APIs?

APIs, or Application Programming Interfaces, are a crucial part of software development. They allow developers to create new applications by leveraging existing functionality from other systems. APIs define how software components should interact and facilitate communication between various products and services without requiring direct implementation.

Importance of APIs

APIs are essential for any engineer because they provide a way to access data and functionality from other systems, which can save time and resources. For instance, APIs can be used to integrate applications into the existing architecture of a server or application, allowing developers to communicate between various products and services without requiring direct implementation.

APIs are also important because they enable developers to create new applications by leveraging existing functionality from other systems. This can help developers throughout the engineering and development process of apps.

APIs are used in a wide range of applications, from social media platforms to e-commerce websites. They are also used in mobile applications, web applications, and desktop applications.

Applications of APIs

APIs have a wide range of applications, some of which are:

1. **Social media platforms:** Social media platforms like Facebook, Twitter, and Instagram use APIs to allow developers to access their data and functionality.

This allows developers to create applications that can interact with these platforms and provide additional functionality to users.

2. **E-commerce websites:** E-commerce websites like Amazon and eBay use APIs to allow developers to access their product catalogs and other data. This allows developers to create applications that can interact with these platforms and provide additional functionality to users.
3. **Weather applications:** Weather applications like AccuWeather and The Weather Channel use APIs to access weather data from various sources. This allows developers to create applications that can provide users with up-to-date weather information.
4. **Maps and navigation applications:** Maps and navigation applications like Google Maps and Waze use APIs to access location data and other information. This allows developers to create applications that can provide users with directions, traffic updates, and other location-based information.
5. **Payment gateways:** Payment gateways like PayPal and Stripe use APIs to allow developers to access their payment processing functionality. This allows developers to create applications that can process payments securely and efficiently.
6. **Messaging applications:** Messaging applications like WhatsApp and Facebook Messenger use APIs to allow developers to access their messaging functionality. This allows developers to create applications that can interact with these platforms and provide additional functionality to users.

Conclusion

In summary, APIs are an essential part of software development, and they provide a way to access data and functionality from other systems. They are used in a wide range of applications and can help developers save time and resources while creating new applications.

REST APIs, Web Scraping, and Working with Files

Web Scraping and HTML Basics

Introduction to web scraping

Web scraping, also known as web harvesting or web data extraction, is the process of extracting information from websites or web pages. It involves automated retrieval of data from web sources. People use it for various applications such as data analysis, mining, price comparison, content aggregation, and more.

How web scraping works

HTTP request

The process typically begins with an HTTP request. A web scraper sends an HTTP request to a specific URL, similar to how a web browser would when you visit a website. The request is usually an HTTP GET request, which retrieves the web page's content.

Web page retrieval

The web server hosting the website responds to the request by returning the requested web page's HTML content. This content includes the visible text and media elements and the underlying HTML structure that defines the page's layout.

HTML parsing

Once the HTML content is received, you need to parse the content. Parsing involves breaking down the HTML structure into components, such as tags, attributes, and text content. You can use BeautifulSoup in Python. It creates a structured representation of the HTML content that can be easily navigated and manipulated.

Data extraction

With the HTML content parsed, web scrapers can now identify and extract the specific data they need. This data can include text, links, images, tables, product prices, news articles, and more. Scrapers locate the data by searching for relevant HTML tags, attributes, and patterns in the HTML structure.

Data transformation

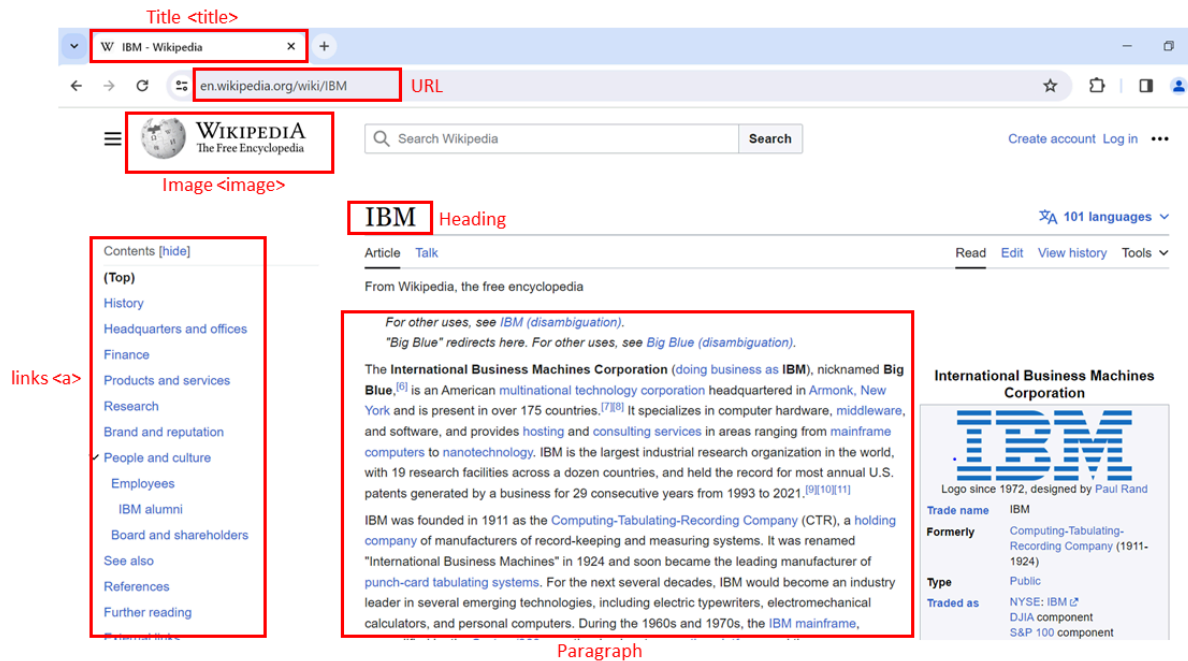
Extracted data may need further processing and transformation. For instance, you can remove HTML tags from text, convert data formats, or clean up messy data. This step ensures the data is ready for analysis or other use cases.

Storage

After extraction and transformation, you can store the scraped data in various formats, such as databases, spreadsheets, JSON, or CSV files. The choice of storage format depends on the specific project's requirements.

Automation

In many cases, scripts or programs automate web scraping. These automation tools allow recurring data extraction from multiple web pages or websites. Automated scraping is especially useful for collecting data from dynamic websites that regularly update their content.



HTML structure

Hypertext markup language (HTML) serves as the foundation of web pages.

Understanding its structure is crucial for web scraping.

- `<html>` is the root element of an HTML page.
- `<head>` contains meta-information about the HTML page.
- `<body>` displays the content on the web page, often the data of interest.
- `<h3>` tags are type 3 headings, making text larger and bold, typically used for player names.
- `<p>` tags represent paragraphs and contain player salary information.

Composition of an HTML tag

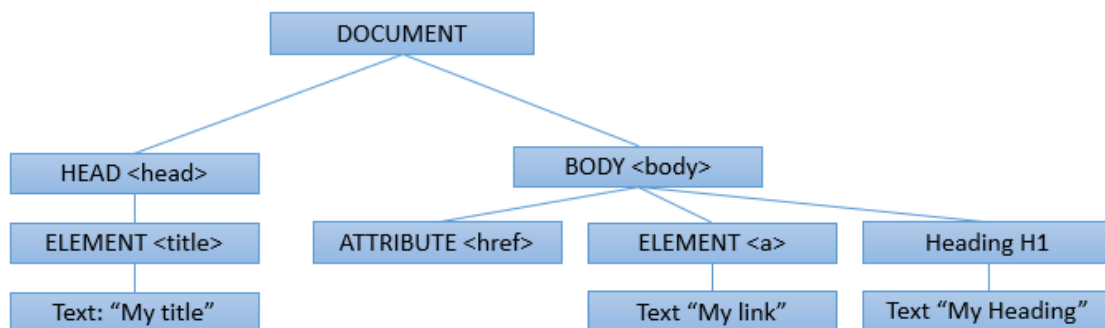
HTML tags define the structure of web content and can contain attributes.

- An HTML tag consists of an opening (start) tag and a closing (end) tag.
- Tags have names (`<a>` for an anchor tag).
- Tags may contain attributes with an attribute name and value, providing additional information to the tag.

HTML document tree

You can visualize HTML documents as trees with tags as nodes.

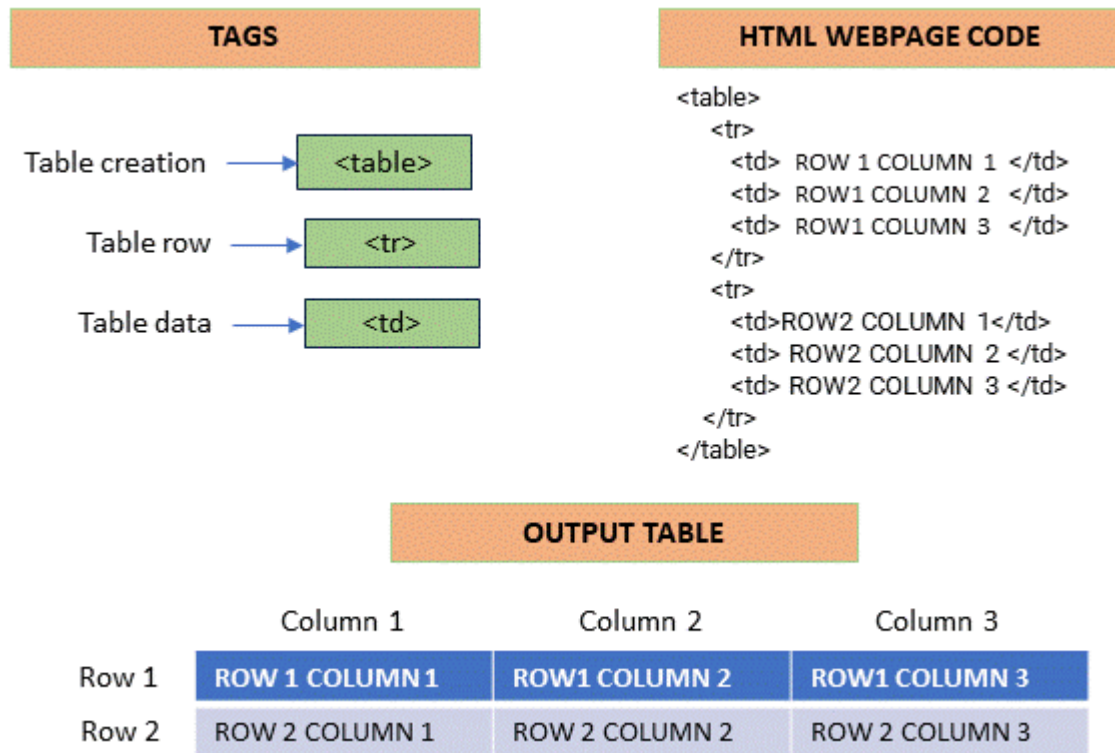
- Tags can contain strings and other tags, making them the tag's children.
- Tags within the same parent tag are considered siblings.
- For example, the `<html>` tag contains both `<head>` and `<body>` tags, making them descendants of `<html>` but children of `<html>`. `<head>` and `<body>` are siblings.



HTML tables

HTML tables are essential for presenting structured data.

- Define an HTML table using the `<table>` tag.
- Each table row is defined with a `<tr>` tag.
- The first row often uses the table header tag, typically `<th>`.
- The table cell is represented by `<td>` tags, defining individual cells in a row.



Web scraping

Web scraping involves extracting information from web pages using Python. It can save time and automate data collection.

Required tools

Web scraping requires Python code and two essential modules: Requests and BeautifulSoup. Ensure you have both modules installed in your Python environment.

```
1 # Import BeautifulSoup to parse the web page content
2 from bs4 import BeautifulSoup
```

Fetching and parsing HTML

To start web scraping, you need to fetch the HTML content of a webpage and parse it using BeautifulSoup. Here's a step-by-step example:

```
1 import requests
2 from bs4 import BeautifulSoup
3
4 # Specify the URL of the webpage you want to scrape
5 url = 'https://en.wikipedia.org/wiki/IBM'
6
7 # Send an HTTP GET request to the webpage
8 response = requests.get(url)
9
10 # Store the HTML content in a variable
11 html_content = response.text
12
13 # Create a BeautifulSoup object to parse the HTML
14 soup = BeautifulSoup(html_content, 'html.parser')
15
16 # Display a snippet of the HTML content
17 print(html_content[:500])
```

Navigating the HTML structure

BeautifulSoup represents HTML content as a tree-like structure, allowing for easy navigation. You can use methods like `find_all` to filter and extract specific HTML elements. For example, to find all anchor tags (`a`) and print their text:

```
1 # Find all <a> tags (anchor tags) in the HTML
2 links = soup.find_all('a')
3
4 # Iterate through the list of links and print their text
5 for link in links:
6     print(link.text)
```

Custom data extraction

Web scraping allows you to navigate the HTML structure and extract specific information based on your requirements. This process may involve finding specific tags, attributes, or text content within the HTML document.

Using BeautifulSoup for HTML parsing

Beautiful Soup is a powerful tool for navigating and extracting specific web page parts. It allows you to find elements based on their tags, attributes, or text, making extracting the information you're interested in easier.

Using pandas read_html for table extraction

Pandas, a Python library, provides a function called `read_html`, which can automatically extract data from websites' tables and present it in a format suitable for analysis. It's similar to taking a table from a webpage and importing it into a spreadsheet for further analysis.

Conclusion

In this reading, you learned about web scraping with BeautifulSoup and Pandas with emphasis on extracting elements and tables. BeautifulSoup facilitates HTML parsing, while Pandas' `read_html` streamlines table extraction. The reading also highlighted responsible web scraping, ensuring adherence to website terms. Armed with this knowledge, you can confidently engage in precise data extraction.

Web Scraping - A Key Tool in Data Science

Introduction

Web scraping, also known as web harvesting or web data extraction, is a technique used to extract large amounts of data from websites. The data on websites is unstructured, and web scraping enables us to convert it into a structured form.

Importance of Web Scraping in Data Science

In the field of data science, web scraping plays an integral role. It is used for various purposes such as:

- **Data Collection:** Web scraping is a primary method of collecting data from the internet. This data can be used for analysis, research, etc.

- **Real-time Application:** Web scraping is used for real-time applications like weather updates, price comparison, etc.
- **Machine Learning:** Web scraping provides the data needed to train machine learning models.

Web Scraping with Python

Python provides several libraries for web scraping. Here are some of them:

BeautifulSoup

BeautifulSoup is a Python library used for web scraping purposes to pull the data out of HTML and XML files. It creates a parse tree from page source code that can be used to extract data in a hierarchical and more readable manner.

```
1 from bs4 import BeautifulSoup
2 import requests
3 URL = "http://www.example.com"
4 page = requests.get(URL)
5 soup = BeautifulSoup(page.content, "html.parser")
```

Scrapy

Scrapy is an open-source and collaborative web crawling framework for Python. It is used to extract the data from the website.

```
1 import scrapy
2 class QuotesSpider(scrapy.Spider):
3     name = "quotes"
4     start_urls = ['http://quotes.toscrape.com/tag/humor/']
5     def parse(self, response):
6         for quote in response.css('div.quote'):
7             yield {'quote': quote.css('span.text::text').get()}
```

Selenium

Selenium is a tool used for controlling web browsers through programs and automating browser tasks.


```
1 from selenium import webdriver
2 driver = webdriver.Firefox()
3 driver.get("http://www.example.com")
```

Applications of Web Scraping

Web scraping is used in various fields and has many applications:

- **Price Comparison:** Services such as ParseHub use web scraping to collect data from online shopping websites and use it to compare the prices of products.
- **Email address gathering:** Many companies that use email as a medium for marketing, use web scraping to collect email ID and then send bulk emails.
- **Social Media Scraping:** Web scraping is used to collect data from Social Media websites such as Twitter to find out what's trending.

Conclusion

Web scraping is an essential skill in the fast-growing world of data science. It provides the ability to turn the web into a source of data that can be analyzed, processed, and used for a variety of applications. However, it's important to remember that one should use web scraping responsibly and ethically, respecting the terms of use or robots.txt files of the websites being scraped.

Web Scraping Tables using Pandas

The Pandas library in Python contains a function `read_html()` that can be used to extract tabular information from any web page.

Consider the following example:

Let us assume we want to extract the list of the largest banks in the world by market capitalization, from the following link:

```
1 URL = 'https://en.wikipedia.org/wiki/List_of_largest_banks'
```

We may use `pandas.read_html()` function in python to extract all the tables in the web page directly.

A snapshot of the webpage is shown below.

The screenshot shows the Wikipedia page for 'List of largest banks'. The page title is 'List of largest banks'. The left sidebar contains a table of contents with links to 'Contents [hide]', '(Top)', 'By market capitalization', 'By total assets', 'Banks by country or territory', 'See also', and 'References'. The main content area has a search bar and a 'Search' button. Below the search bar, the page title 'List of largest banks' is displayed. The article text states: 'From Wikipedia, the free encyclopedia. The following are lists of the largest banks in the world, as measured by market capitalization. By market capitalization [edit]. The list is based on Forbes.com's ranking as of August 2023 based on an analysis of the bank the global economy.[1]'. A table is shown with 3 columns: Rank, Bank name, and Market cap [hide] (US\$ billion). The table lists the top 10 banks by market capitalization as of August 2023.

Rank	Bank name	Market cap [hide] (US\$ billion)
1	JPMorgan Chase	419.25
2	Bank of America	231.52
3	Industrial and Commercial Bank of China	194.56
4	Agricultural Bank of China	160.68
5	HDFC Bank	157.91
6	Wells Fargo	155.87
7	HSBC Holdings PLC	148.90
8	Morgan Stanley	140.83
9	China Construction Bank	139.82
10	Bank of China	136.81

We can see that the required table is the first one in the web page.

Note: This is a live web page and it may get updated over time. The image shown above has been captured in November 2023. The process of data extraction remains the same.

We may execute the following lines of code to extract the required table from the web page.

```
1 import pandas as pd
2 URL = 'https://en.wikipedia.org/wiki/List_of_largest_banks'
3 tables = pd.read_html(URL)
4 df = tables[0]
5 print(df)
```

This will extract the required table as a dataframe `df`. The output of the print statement would look as shown below.

	Rank	Bank name	Market cap(US\$ billion)
0	1	JPMorgan Chase	419.25
1	2	Bank of America	231.52
2	3	Industrial and Commercial Bank of China	194.56
3	4	Agricultural Bank of China	160.68
4	5	HDFC Bank	157.91
5	6	Wells Fargo	155.87
6	7	HSBC Holdings PLC	148.90
7	8	Morgan Stanley	140.83
8	9	China Construction Bank	139.82
9	10	Bank of China	136.81

Although convenient, this method comes with its own set of limitations. Firstly, web pages may have content saved in them as tables but they may not appear as tables on the web page.

For instance, consider the following URL showing the list of countries by GDP (nominal).

```
1 URL = 'https://en.wikipedia.org/wiki/List\_of\_countries\_by\_GDP\_\(nominal\)'
```

The images on the web page are also saved in tabular format. A snapshot of the web page is shared below.

List of countries by GDP (nominal)

75 languages

Article Talk

Read View source View history Tools

From Wikipedia, the free encyclopedia

For countries by GDP based on purchasing power parity, see *List of countries by GDP (PPP)*.

For countries by GDP per capita, see *List of countries by GDP (nominal) per capita*.

Gross domestic product (GDP) is the market value of all final goods and services from a nation in a given year.^[2] Countries are sorted by nominal GDP estimates from financial and statistical institutions, which are calculated at market or government official exchange rates. Nominal GDP does not take into account differences in the cost of living in different countries, and the results can vary greatly from one year to another based on fluctuations in the exchange rates of the country's currency.^[2] Such fluctuations may change a country's ranking from one year to the next, even though they often make little or no difference in the standard of living of its population.^[4]

Comparisons of national wealth are also frequently made on the basis of purchasing power parity (PPP), to adjust for differences in the cost of living in different countries. Other metrics, nominal GDP per capita and a corresponding GDP (PPP) per capita are used for comparing national standard of living. On the whole, PPP per capita figures are less spread than nominal GDP per capita figures.^[4]

The rankings of national economies over time have changed considerably; the United States surpassed the British Empire's output around 1916,^[5] which in turn had surpassed the Qing dynasty in aggregate output decades earlier.^{[7][8]} Since China's transition to a socialist market economy through controlled privatisation and deregulation,^{[9][10]} the country has seen its ranking increase from ninth in 1978, to second in 2010; China's economic growth accelerated during this period and its share of global nominal GDP surged from 2% in 1980 to 18% in 2021.^{[10][11]} Among others, India has also experienced an economic boom since the implementation of economic liberalisation in the early 1990s.^[12]

The first list includes estimates compiled by the *International Monetary Fund's* *World Economic Outlook*, the second list shows the *World Bank's* data, and the third list includes data compiled by the *United Nations Statistics Division*. The IMF definitive data for the past year and estimates for the current year are published twice a year in April and October. Non-sovereign entities (the world, continents, and some *dependent territories*) and states with limited international recognition (such as *Kosovo* and *Taiwan*) are included in the list where they appear in the sources.

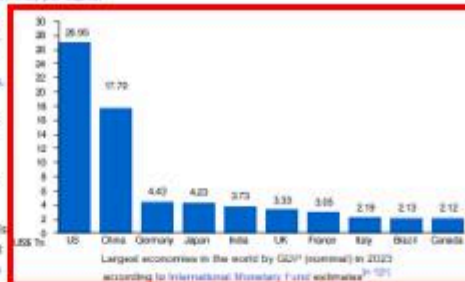


Table 1



Table 2

Table

The table initially ranks each country or territory with their latest available estimates, and can be reranked by either of the sources

The links in the "Country/Territory" row of the following table link to the article on the GDP or the economy of the respective country or territory.

GDP (USD million) by country								
	Country/Territory	UN region	IMF ^{[1][1]}	World Bank ^[14]	United Nations ^[15]			
			Forecast	Year	Estimate	Year	Estimate	Year
	World	—	104,476,452	2023	100,662,011	2022	86,888,006	2021
1	 United States	Americas	26,949,643	2023	26,462,700	2022	23,315,081	2021
2	 China	Asia	17,700,899	^[1] 2023	17,963,171	^[1] 2022	17,734,131	^[1] 2021
3	 Germany	Europe	4,429,838	2023	4,072,192	2022	4,259,935	2021
4	 Japan	Asia	4,230,862	2023	4,231,141	2022	4,940,878	2021
5	 India	Asia	3,732,224	2023	3,385,090	2022	3,201,471	2021
6	 United Kingdom	Europe	3,332,059	2023	3,070,668	2022	3,131,378	2021
7	 France	Europe	3,049,016	2023	2,782,505	2022	2,967,880	2021
8	 Italy	Europe	2,186,062	2023	2,010,432	2022	2,107,703	2021
9	 Brazil	Americas	2,126,805	2023	1,920,096	2022	1,608,981	2021
10	 Canada	Americas	2,117,805	2023	2,129,840	2022	1,988,336	2021
11	 Russia	Europe	1,862,470	2023	2,240,422	2022	1,778,782	2021
12	 Mexico	Americas	1,811,468	2023	1,414,187	2022	1,272,839	2021
13	 South Korea	Asia	1,709,232	2023	1,665,246	2022	1,810,966	2021
14	 Australia	Oceania	1,687,713	2023	1,675,419	2022	1,734,532	2021
15	 Spain	Europe	1,582,054	2023	1,397,509	2022	1,427,381	2021
16	 Indonesia	Asia	1,417,387	2023	1,319,100	2022	1,186,093	2021
17	 Turkey	Asia	1,154,600	2023	905,988	2022	819,034	2021
18	 Netherlands	Europe	1,003,319	2023	884,445	2022	6,543,812	2021

Table 3

Secondly, the contents of the tables in the web pages may contain elements such as hyperlink text and other denoters, which are also scraped directly using the pandas

method. This may lead to a requirement of further cleaning of data. A closer look at table 3 in the image shown above indicates that there are many hyperlink texts which are also going to be treated as information by the pandas function.

GDP (USD million) by country								
	Country/Territory	UN region	IMF ^[1] ^[13]		World Bank ^[14]		United Nations ^[15]	
			Forecast	Year	Estimate	Year	Estimate	Year
	World	—	104,476,432	2023	100,562,011	2022	96,698,005	2021
1	 United States	Americas	26,949,643	2023	25,462,700	2022	23,315,081	2021
2	 China	Asia	17,700,899	^[n 1] 2023	17,963,171	^[n 3] 2022	17,734,131	^[n 1] 2021
3	 Germany	Europe	4,429,838	2023	4,072,192	2022	4,259,935	2021
4	 Japan	Asia	4,230,862	2023	4,231,141	2022	4,940,878	2021
5	 India	Asia	3,732,224	2023	3,385,090	2022	3,201,471	2021
6	 United Kingdom	Europe	3,332,059	2023	3,070,668	2022	3,131,378	2021
7	 France	Europe	3,049,016	2023	2,782,905	2022	2,957,880	2021
8	 Italy	Europe	2,186,082	2023	2,010,432	2022	2,107,703	2021
9	 Brazil	Americas	2,126,809	2023	1,920,096	2022	1,608,981	2021
10	 Canada	Americas	2,117,805	2023	2,139,840	2022	1,988,336	2021
11	 Russia	Europe	1,862,470	2023	2,240,422	2022	1,778,782	2021
12	 Mexico	Americas	1,811,468	2023	1,414,187	2022	1,272,839	2021
13	 South Korea	Asia	1,709,232	2023	1,665,246	2022	1,810,966	2021
14	 Australia	Oceania	1,687,713	2023	1,675,419	2022	1,734,532	2021
15	 Spain	Europe	1,582,054	2023	1,397,509	2022	1,427,381	2021

We can extract the table using the code shown below.

```

1 import pandas as pd
2 URL = 'https://en.wikipedia.org/wiki/List_of_countries_by_GDP_(nominal)'
3 tables = pd.read_html(URL)
4 df = tables[2] # the required table will have index 2
5 print(df)

```

The output of the print statement is shown below.

	Country/Territory	UN region	IMF[1][13]		World Bank[14]		United Nations[15]	
	Country/Territory	UN region	Forecast	Year	Estimate	Year	Estimate	Year
0	World	-	104476432	2023	100562011	2022	96698005	2021
1	United States	Americas	26949643	2023	25462700	2022	23315081	2021
2	China	Asia	17700899	[n 1]2023	17963171	[n 3]2022	17734131	[n 1]2021
3	Germany	Europe	4429838	2023	4072192	2022	4259935	2021
4	Japan	Asia	4230862	2023	4231141	2022	4940878	2021
..	...	...	...	...	...	...	...	...
209	Palau	Oceania	267	2023	-	-	218	2021
210	Kiribati	Oceania	246	2023	223	2022	227	2021
211	Nauru	Oceania	150	2023	151	2022	155	2021
212	Montserrat	Americas	-	-	-	-	72	2021
213	Tuvalu	Oceania	63	2023	60	2022	60	2021

Note that the hyperlink texts have also been retained in the code output.

It is further prudent to point out, that this method exclusively operates only on tabular data extraction. BeautifulSoup library still remains the default method of extracting any kind of information from web pages.

Module 5 Summary: APIs and Data Collection

- Simple APIs in Python are application programming interfaces that provide straightforward and easy-to-use methods for interacting with services, libraries, or data, often with minimal configuration or complexity.
 - An API lets two pieces of software talk to each other.
 - Using an API library in Python entails importing the library, calling its functions or methods to make HTTP requests, and parsing the responses to access data or services provided by the API.
 - Pandas API processes the data by communicating with the other software components.
 - An Instance forms when you create a dictionary and then use the DataFrames constructor to create a Pandas object.
 - Method “head()” will display the mentioned number of rows from the top (default 5) of DataFrames, while method “mean()” will calculate the mean and return the values

- Rest APIs allow you to communicate through the internet, taking advantage of resources like storage, access more data, AI algorithms, and so on.
 - HTTP methods transmit data over the internet.
 - An HTTP message typically includes a JSON file with instructions for operations.
 - HTTP messages containing JSON files are returned to the client as a response from web services.
 - Dealing with time series data involves using the Pandas time series function.
 - You can get data for daily candlesticks and plot the chart using Plotly with the candlestick plot.
- The HTTP (HyperText Transfer Protocol) transfers data, including web pages and resources, between a client (a web browser) and a server on the World Wide Web.
 - The HTTP protocol is commonly used for implementing various types of REST APIs.
 - An HTTP response includes information like the type of resource, length of resource, and so on
 - Uniform resource locator (URL) is the most popular way to find resources on the web.
 - URL is divided into three parts: scheme, internet address or base URL, and route
 - The GET method is one of the popular methods of requesting information. Some other methods may also include the body.
 - Response method contains the version and body of the response.
 - POST submits data to the server, PUT updates data already on the server, DELETE deletes data from the server
- Requests is a Python library that allows you to send HTTP/1.1 requests easily
 - You can modify the results of your query with the GET method.
 - You can obtain multiple requests from a URL like name, ID, and so on with a Query string.
- Web scraping in Python involves extracting and parsing data from websites to gather information for various applications, using libraries like BeautifulSoup and requests.

- HTML comprises text surrounded by blue text elements enclosed in angular brackets called tags.
- You can select an HTML element on a web page to inspect the webpage.
- Web pages may also contain CSS and JavaScript along with HTML elements.
- Each HTML document is like an HTML Tree, which may contain strings and other tags.
- Each HTML table is comprised of table tags and is structured with elements such as rows, headers, body and so on.
- Tabular data can also be extracted from web pages using the ``read_html`` method in Pandas.
- BeautifulSoup in Python is a library for parsing and navigating HTML and XML documents, making extracting, and manipulating data from web pages more accessible.
- To parse a document, pass it through the BeautifulSoup constructor to get a beautiful soup object representing the document as a nested data structure.
- BeautifulSoup represents HTML as a set of tree-like objects with methods to parse the HTML.
- Navigable string is like a Python string that supports beautiful soup functionality.
- `find_all` is a method used to extract content based on the tag's name, its attributes, the text of a string, or some combination of these.
- The `find_all` method looks through a tag's descendants and retrieves all descendants that match your filters.
- The result is a Python iterable like a list.
- File formats refer to the specific structure and encoding rules used to store and represent data in files, such as `.txt` for plain text or `.csv` for comma-separated values.
- Python works with different file formats such as CSV, XML, JSON, `xlsx`, and so on
- The extension of a file name will let you know what type of file it is and what it needs to open with.
- To access data from CSV files, we can use Python libraries such as Pandas.

- Similarly, different methods help parse JSON, XML, and other files.

Cheat Sheet: APIs and Data Collection

Use this cheat sheet for various packages/methods, their descriptions, and examples.

Accessing element attribute

Access the value of a specific attribute of an HTML element.

Syntax:

```
1 attribute = element[(attribute)]
```

Example:

```
1 href = link_element[(href)]
```

BeautifulSoup()

Parse the HTML content of a web page using BeautifulSoup. The parser type can vary based on the project.

Syntax:

```
1 soup = BeautifulSoup(html, (html.parser))
```

Example:

```
1 html = (https://api.example.com/data) soup = BeautifulSoup(html, (html.parser))
```

delete()

Send a DELETE request to remove data or a resource from the server. DELETE requests delete a specified resource on the server.

Syntax:

```
1 response = requests.delete(url)
```

Example:

```
1 response = requests.delete(https://api.example.com/delete)
```

find()

Find the first HTML element that matches the specified tag and attributes.

Syntax:

```
1 element = soup.find(tag, attrs)
```

Example:

```
1 first_link = soup.find((a), {(class): (link)})
```

find_all()

Find all HTML elements that match the specified tag and attributes.

Syntax:

```
1 elements = soup.find_all(tag, attrs)
```

Example:

```
1 all_links = soup.find_all((a), {(class): (link)})</td>
```

findChildren()

Find all child elements of an HTML element.

Syntax:

```
1 children = element.findChildren()
```

Example:

```
1 child_elements = parent_div.findChildren()
```

get()

Perform a GET request to retrieve data from a specified URL. GET requests are typically used for reading data from an API. The response variable will contain the server's response, which you can process further.

Syntax:

```
1 response = requests.get(url)
```

Example:

```
1 response = requests.get((https://api.example.com/data))
```

Headers

Include custom headers in the request. Headers can provide additional information to the server, such as authentication tokens or content types.

Syntax:

```
1 headers = {(HeaderName): (Value)}
```

Example:

```
1. base_url = (https://api.example.com/data) headers =  
  {(Authorization): (Bearer YOUR_TOKEN)} response =  
  requests.get(base_url, headers=headers)
```

Import Libraries

Import the necessary Python libraries for web scraping.

Syntax:

```
1 from bs4 import BeautifulSoup
```

json()

Parse JSON data from the response. This extracts and works with the data returned by the API. The `response.json()` method converts the JSON response into a Python data structure, usually a dictionary or list.

Syntax:

```
1 data = response.json()
```

Example:

```
1 response = requests.get((https://api.example.com/data))
2 data = response.json()
```

next_sibling()

Find the next sibling element in the DOM.

Syntax:

```
1 sibling = element.find_next_sibling()
```

Example:

```
1 next_sibling = current_element.find_next_sibling()
```

parent

Access the parent element in the Document Object Model (DOM).

Syntax:

```
1 parent = element.parent
```

Example:

```
1 parent_div = paragraph.parent
```

post()

Send a POST request to a specified URL with data. Create or update POST requests using resources on the server. The data parameter contains the data to send to the server, often in JSON format.

Syntax:

```
1 response = requests.post(url, data)
```

Example:

```
1 response = requests.post((https://api.example.com/submit), data={{key}: (value)})
```

put()

Send a PUT request to update data on the server. PUT requests are used to update an existing resource on the server with the data provided in the data parameter, typically in JSON format.

Syntax:

```
1 response = requests.put(url, data)
```

Example:

```
1 response = requests.put(https://api.example.com/update, data={(key): (value)})
```

Query parameters

Pass query parameters in the URL to filter or customize the request. Query parameters specify conditions or limits for the requested data.

Syntax:

```
1 params = {(param_name): (value)}
```

Example:

```
1 base_url = "https://api.example.com/data"
2 params = {"page": 1, "per_page": 10}
3 response = requests.get(base_url, params=params)
```

select()

Select HTML elements from the parsed HTML using a CSS selector.

Syntax:

```
1 element = soup.select(selector)
```

Example:

```
1 titles = soup.select((h1))
```

status_code

Check the HTTP status code of the response. The HTTP status code indicates the result of the request (success, error, redirection). It can be used for error handling and decision-making in your code.

Syntax:

```
1 response.status_code
```

Example:

```
1 url = "https://api.example.com/data"  
2 response = requests.get(url)  
3 status_code = response.status_code
```

tags for find() and find_all()

Specify any valid HTML tag as the tag parameter to search for elements of that type. Here are some common HTML tags that you can use with the tag parameter.

Tag examples:

```
1 - (a): Find anchor () tags.
2 - (p): Find paragraph ((p)) tags.
3 - (h1), (h2), (h3), (h4), (h5), (h6): Find heading tags from level 1 to 6 ( (h1),n (h2)).
4 - (table): Find table () tags.
5 - (tr): Find table row () tags.
6 - (td): Find table cell ((td)) tags.
7 - (th): Find table header cell ((td))tags.
8 - (img): Find image ((img)) tags.
9 - (form): Find form ((form)) tags.
10 - (button): Find button ((button)) tags.
```

text

Retrieve the text content of an HTML element.

Syntax:

```
1 text = element.text
```

Example:

```
1 title_text = title_element.text
```

Module 5 Glossary: APIs and Data Collection

Term	Definition
------	------------

API Key	An API key in Python is a secure access token or code used to authenticate and authorize access to an API or web service, enabling the user to make authenticated requests.
APIs	APIs (Application Programming Interfaces) are a set of rules and protocols that enable different software applications to communicate and interact, facilitating the exchange of data and functionality.
Audio file	An audio file is a digital recording or representation of sound, often stored in formats like MP3, WAV, or FLAC, allowing playback and storage of audio content.
Authorize	In Python, "authorize" often means granting permission or access to a user or system to perform specific actions or access particular resources, often related to authentication and authorization mechanisms.
Beautiful Soup Objects	Beautiful Soup objects in Python are representations of parsed HTML or XML documents, allowing easy navigation, searching, and manipulation of the document's elements and data.
Bitcoin currency	Bitcoin is a decentralized digital currency that operates without a central authority, allowing peer-to-peer transactions on a blockchain network.
Browser	A browser is a software application that enables users to access and interact with web content, displaying websites and web applications.
Candlestick plot	A candlestick plot in Python visually represents stock price movements over time, using rectangles to illustrate the open, close, high, and low prices for a given period.

Client/Wrapper	A client or wrapper in Python is a software component that simplifies interaction with external services or APIs, encapsulating communication and providing higher-level functionality for developers.
CoinGecko API	The CoinGecko API is a web service that provides cryptocurrency market data and information, allowing developers to access real-time and historical data for various cryptocurrencies.
DELETE Method	The DELETE method in Python is an HTTP request method used to request the removal or deletion of a resource on a web server.
Endpoint	In Python, an "endpoint" refers to a specific URL or URI that a web service or API exposes to perform a particular function or access a resource.
File extension	A file extension is a suffix added to a filename to indicate the file's format or type, often used by operating systems and applications to determine how to handle the file.
find_all	In Python, find_all is a BeautifulSoup method used to search and extract all occurrences of a specified HTML or XML element, returning a list of matching elements.
GET method	The GET method in Python is an HTTP request method used to retrieve data from a web server by appending parameters to the URL.
HTML	HTML (Hypertext Markup Language) is the standard language for creating and structuring content on web pages, using tags to define the structure and presentation of documents.

HTML Anchor tags	HTML anchor tags in Python are used to create hyperlinks within web pages, linking to other web pages or resources using the <a> element with the href attribute.
HTML Tables	HTML tables in Python are used to organize and display data in a structured grid format on a web page, constructed with <table>, <tr>, <th>, and <td> elements.
HTML Tag	An HTML tag in Python is a specific code enclosed in angle brackets used to define elements within an HTML document, specifying how content should be presented or structured.
HTML Trees	HTML trees in Python refer to the hierarchical structure created when parsing an HTML document, representing its elements and their relationships, typically used for manipulation or extraction of data.
HTTP	HTTP (HyperText Transfer Protocol) is the foundation of data communication on the World Wide Web, used for transmitting and retrieving web content between clients and servers.
httplib	A library that provides a set of functions and classes to send and handle HTTP and HTTPS requests.
Identify	In Python, "identify" usually means determining if two variables or objects refer to the same memory location, which can be checked using the is operator.
Instance	In Python, an "instance" typically refers to a specific occurrence of an object or class, created from a class blueprint, with its own unique set of data and attributes.

JSON file	A JSON (JavaScript Object Notation) file is a lightweight data interchange format that stores structured data in a human-readable text format, commonly used for configuration, data exchange, and web APIs.
Mean value	The mean value in Python is the average of a set of numerical values, calculated by adding all values and dividing by the total number of values.
Navigable string	In Python, a Navigable String is a BeautifulSoup object representing a string within an HTML or XML document, allowing for navigation and manipulation of the text content.
Plotly	Plotly is a Python library for creating interactive and visually appealing web-based data visualizations and dashboards.
PNG file	A PNG (Portable Network Graphics) file is a lossless image format in Python that is commonly used for high-quality graphics with support for transparency and compression.
POST method	The POST method in Python is an HTTP request method used to send data to a web server, often used for submitting form data and creating or updating resources.
Post request	A POST request in Python is an HTTP method used to send data to a web server for the purpose of creating or updating a resource, typically used in web applications and APIs.
PUT method	The PUT method in Python is an HTTP request method used to update an existing resource on a web server by replacing or modifying it.

Py-Coin-Gecko	Py-Coin-Gecko is a Python library that provides a convenient interface for accessing cryptocurrency data and information from the CoinGecko API.
Python iterable	A Python iterable is an object that can be looped over, typically used in for loops, and includes data structures like lists, tuples, and dictionaries.
Query string	A query string in Python is a part of a URL that contains data or parameters to be sent to a web server, typically used in HTTP GET requests to retrieve specific information.
rb mode	In Python, "rb" mode is used when opening a file to read it in binary mode, allowing you to read and manipulate non-text files like images or binary data.
Resource	In Python, a "resource" typically refers to an external entity such as a file, database connection, or network object that can be managed and manipulated within a program.
Rest API	A REST API in Python is a web-based interface that follows the principles of Representational State Transfer (REST), allowing communication and data exchange over HTTP using standard HTTP methods and data formats.
Service instance	In Python, a "service instance" typically refers to an instantiated object or entity representing a service, enabling interaction with that service in a program or application.
Timestamp	A timestamp is a representation of a specific moment in time, often expressed as a combination of date and time, used for record-keeping and data tracking.

Transcribe	"Transcribe" typically means converting spoken language or audio into written text, often using automatic speech recognition (ASR) technology.
Unix timestamp	A UNIX timestamp is a numerical value representing the number of seconds that have elapsed since January 1, 1970, 00:00:00 UTC, used for time-keeping in Unix-based systems and programming.
url (Uniform Resource Locator)	In Python, a URL (Uniform Resource Locator) is a web address that specifies the location of a resource on the internet, typically consisting of a protocol, domain, and path.
urllib	The "urllib" library in Python is used for working with URLs and making HTTP requests, including functions for fetching web content, handling cookies, and more.
Web service	Web services in Python are software components that allow applications to communicate over the internet by sending and receiving data in a standardized format, typically using protocols like HTTP or XML.
Web scraping	Web scraping in Python is the process of extracting data from websites by parsing and analyzing their HTML structure, often done with libraries like BeautifulSoup or Scrapy.
xlsx	An XLSX file is a file format used for storing spreadsheet data in Excel, containing worksheets, cells, and formulas in a structured manner.

xml

XML (Extensible Markup Language) is a text-based format for storing and structuring data using tags, often used for data interchange and configuration files.